LeAH: Generalizing Test-Time Agent Harness Adaptation to Unseen Environments

Zekun Wang[1,*], **Yingqiang Ge**[2], **Hai Wang**[2], **Yuhui Xu**[2], **Joshua Frandsen**[2], **Eric Yang**[2], **Shancong Fu**[2], **Christopher J. MacLellan**[1], **Chandan K. Reddy**[2]

[1]Georgia Institute of Technology [2]Amazon

Abstract

A language model agent acts through a harness of prompts and code, and a harness that works in one environment can fail in another. Harness evolution searches for a better harness on each benchmark, and trained harness proposers depend on the benchmarks or runtime contexts they were trained on. We study how to learn harness adaptation that generalizes to new benchmarks: a generalist proposer, meta-trained across agent benchmarks, reads a few failures of an unseen benchmark and writes one edit that improves the agent's harness there, without search or weight updates. Rewarding an edit on its own benchmark can favor edits that only suit the training benchmarks, so we formulate generalization as a joint objective: an edit should succeed on its benchmark and teach the proposer to write a successful edit for another benchmark. Under a hierarchical Bayesian view, the proposer's learning from an edit can be estimated by in-context learning, which turns the objective into a weighted supervised loss. We call the resulting method Learning to Adapt Harnesses (LeAH). We train a 9B proposer with supervised fine-tuning (SFT) on a teacher model's demonstrations for three agent benchmarks and then meta-train it with LeAH. On held-out tasks of these benchmarks and on six unseen benchmarks, its edits improve a frozen agent over the initial harness and baselines, exceeding its teacher on the training benchmarks and matching it on the unseen ones. On the unseen benchmarks, LeAH raises the average benchmark score, on a 0 to 100 scale, from 51.4 under the initial harness to 57.9, while the same training with each edit rewarded only on its own benchmark reaches 53.5. The edits of the proposer trained this way more often carry over forms from its training benchmarks, while those of LeAH more often use the rules of the new benchmark.

1 Introduction

Humans can adapt to a new task from feedback about a few mistakes, as prior experience narrows the changes worth considering (Braun et al., 2009; Collins & Frank, 2013; Harlow, 1949; Kemp et al., 2007). An agent should likewise use a few failures in a new environment to improve on its other tasks there. A language model agent, which we call the executor, acts through a harness of prompts, tools and control flow, and a harness found for one benchmark can fall below the initial harness on another (Wang et al., 2026b). For example, a hook that warns the executor when it repeats a cycle of actions can read the actions from the tool-call field on benchmarks where the executor acts through tool calls, but on TextWorld the executor writes its actions as plain text, so the field stays empty and the warning never appears (Section 5.4).

Harness evolution searches for a better harness on each benchmark over hundreds to thousands of task executions (Lee et al., 2026; Lin et al., 2026; Wang et al., 2026b). Trained proposers, language models that write harness edits, are trained separately for each benchmark or during a search on it (Singhal et al., 2026; Sagar et al., 2026; Zhang et al., 2026d; Ma et al., 2026; Xu et al., 2026), or edit through a runtime context written for each of their benchmarks (Shao et al., 2026). In each case, what adapts the harness comes from the benchmark itself. We therefore ask whether a proposer

*Work done during an internship at Amazon. Correspondence to: zekun@gatech.edu.

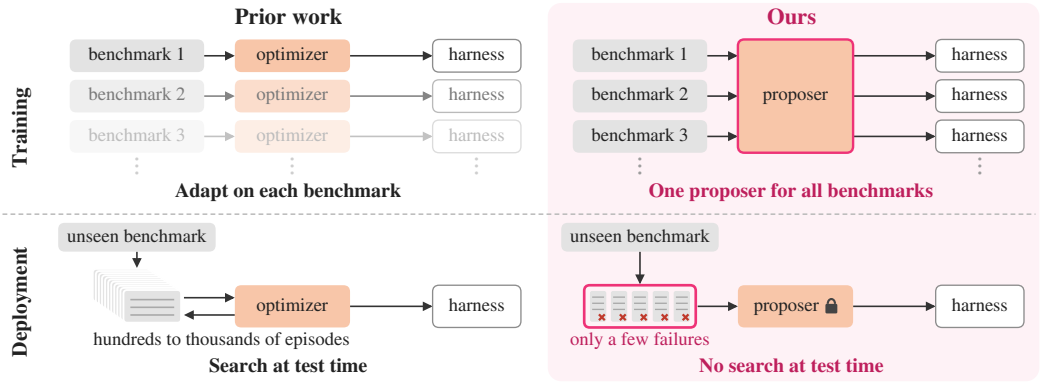


Figure 1: Prior work and ours. Harness evolution (Lee et al., 2026; Lin et al., 2026; Wang et al., 2026b; Nie et al., 2026) and trained proposers (Ma et al., 2026; Xu et al., 2026) adapt on each benchmark and search at test time. LeAH trains one proposer on all training benchmarks, and at deployment the proposer writes one edit from a few failures of a new benchmark.

can learn an adaptation rule that generalizes, so that from a few failures of a new benchmark it improves the executor on that benchmark.

We study this question through meta-learning, training one generalist proposer across agent benchmarks. A new environment offers little data, so adaptation here is one free-form edit to the executor's harness, written from a few failure trajectories and rewarded by how much it improves the executor on other tasks of the same benchmark. Since this reward can be raised by edits that suit only the training benchmarks, we formulate generalization as a joint objective: an edit should succeed on its own benchmark, and after the proposer learns from it, the edit it writes for another benchmark should also succeed. Under a hierarchical Bayesian view, in which one latent fix strategy of the proposer serves all benchmarks (Grant et al., 2018), learning from an edit is a posterior update that in-context learning can estimate (Xie et al., 2022), and this turns the objective into a weighted supervised loss on adaptation and transfer examples. We call the resulting method Learning to Adapt Harnesses (LeAH). At deployment, the proposer adapts the harness to a new benchmark from a few of its failures, with no search, weight update, runtime context or demonstrations for that benchmark (Figure 1).

We train a 9B proposer with supervised fine-tuning (SFT) on a teacher model's demonstrations for ALFWorld, DBBench and WebShop, following prior work (Liu et al., 2024; Zeng et al., 2024; Shao et al., 2026), and then meta-train it with LeAH. It improves the executor over the initial harness on held-out tasks of these benchmarks, where it is above its teacher, and on six unseen benchmarks, where it matches its teacher while the SFT proposer and Harness-R1 fall below the initial harness. Training without transfer examples, which we call ordinary reinforcement learning, gains about a third as much on the unseen benchmarks, and its edits more often carry over forms from its training benchmarks, while those of LeAH more often use the rules of the new benchmark. Our contributions are:

- We cast harness adaptation as meta-learning across agent benchmarks, in which one generalist proposer learns to adapt a harness to a new benchmark from a few failure trajectories, with one edit and no search or weight update.
- We formulate generalization across benchmarks as a joint objective: an edit should succeed on its own benchmark and teach the proposer to write a successful edit for another. Under a hierarchical Bayesian view, in-context learning can estimate how the proposer learns from an edit. From this estimate we derive LeAH, which optimizes the objective with a weighted supervised loss.
- We evaluate LeAH on held-out tasks of three training benchmarks and on six unseen benchmarks. It improves the executor over the initial harness on both, and reaches a higher average on the unseen benchmarks than ordinary reinforcement learning, with comparable success on the training ones. Our analysis shows that its edits more often use the rules and action interface of the new benchmark.

2 Related Work

Meta-learning. Meta-learning aims to learn from a distribution of tasks so that a learner can adapt to a new task from limited evidence. A common objective minimizes query loss after adaptation on a support set, learning an initialization, representation or update rule (Finn et al., 2017; Snell et al., 2017; Li et al., 2017). The adaptation learned this way need not generalize beyond the distribution of the training tasks, and on new tasks from another domain, meta-learned few-shot classifiers can fall below simple fine-tuning (Chen et al., 2019; Guo et al., 2020). MLDG meta-learns for generalization to new domains by splitting the source domains into meta-train and meta-test domains, so that a gradient step that improves performance on the meta-train domains must also improve it on the meta-test domains (Li et al., 2018). Recent language model methods meta-train a model to learn new tasks from examples in its context (Min et al., 2022), learn token weights for fine-tuning (Hu et al., 2023), write instructions for unseen classification datasets from labeled examples (Xiao et al., 2026), meta-learn a workflow initialization that language model feedback refines for each task (Zhu et al., 2025), or learn from reasoning and interaction feedback (Qu et al., 2025; Jiang et al., 2026). The hierarchical Bayesian view of meta-learning treats the tasks as sharing latent parameters that each support set updates (Baxter, 1997; Grant et al., 2018). In our setting, a benchmark is a task, its support set is a few failure trajectories, and the adaptation is an edit that the proposer writes in context for the harness of a separate executor, which we model under this view. Like MLDG, LeAH meta-learns for generalization, but the proposer learns from an edit through a posterior update estimated in context, since a gradient step on every sampled edit is costly for a large model.

Harness evolution. Harness evolution uses execution feedback to improve an executor's prompts, tools and control flow, either by evolving the executor's harness directly or by learning the harness optimizer, a proposer that edits the executor's harness. Search methods repeatedly propose and evaluate revisions of a harness program (Lee et al., 2026; Lin et al., 2026), reusing failure summaries and edit histories (Zhang et al., 2026c; Chen et al., 2026; Park et al., 2026) or letting the optimizer verify its edits and revise its own harness (Wang et al., 2026b). Prompt optimization runs the same search over instructions (Zhou et al., 2023; Yang et al., 2024; Pryzant et al., 2023; Agrawal et al., 2026), and TTHE runs it at test time on the agent's own traces (Nie et al., 2026). These searches run again on each new benchmark. Trained proposers are learned harness optimizers that carry an adaptation rule in their parameters, but most serve one benchmark or task: they write prompts for the benchmark they were trained on (Singhal et al., 2026), build a harness for each query from a fixed set of actions (Sagar et al., 2026), patch a harness already searched for the benchmark (Zhang et al., 2026d), generate harnesses for individual tasks within a fixed protocol (Zhang et al., 2026b), or train during a search on the target task (Ma et al., 2026; Xu et al., 2026). Retroformer trains a model with proximal policy optimization to write reflections for a frozen agent, rated by the change in return they produce, but each reflection is written for the next attempt at the same task (Yao et al., 2024). Harness-R1 learns edits from failure evidence across three agent benchmarks, but its hooks act through runtime state and predicates written for each of them, its reward uses the task batch that supplied the failures, and its transfer experiments retain the training benchmarks (Shao et al., 2026). Our proposer is rewarded on query tasks disjoint from its failures and writes a free-form edit from a few failures, with no runtime context written for the benchmark and no search.

Generalization in harness evolution. Harness evolution often tests generalization by changing the model, running a harness found with one executor on other executors (Lee et al., 2026; Lin et al., 2026) or rerunning the method with other models (Zhang et al., 2026c; Nie et al., 2026). HarnessTTT moves its harness to a stronger executor on the same task and continues the search, so these gains also reflect the stronger executor (Ma et al., 2026). Transfer to new tasks is measured on held-out tasks of the same benchmark, where a controlled study (Wang et al., 2026a) finds no average gain for Agentic Harness Engineering (Lin et al., 2026), or on a few related benchmarks (Lin et al., 2026; Xu et al., 2026; Singhal et al., 2026), where an evolved harness can fall below the initial harness of the new benchmark (Wang et al., 2026b). Closest to our setting, Harness Learning trains one proposer across 24 Reasoning Gym task families, and its single revisions exceed the seed harness on average over 21 unseen families of the same suite, with a separate proposer for question answering (Zhang et al., 2026a). In these methods, generalization to other benchmarks is measured after the search or the training, but it is not part of what they optimize. LeAH makes

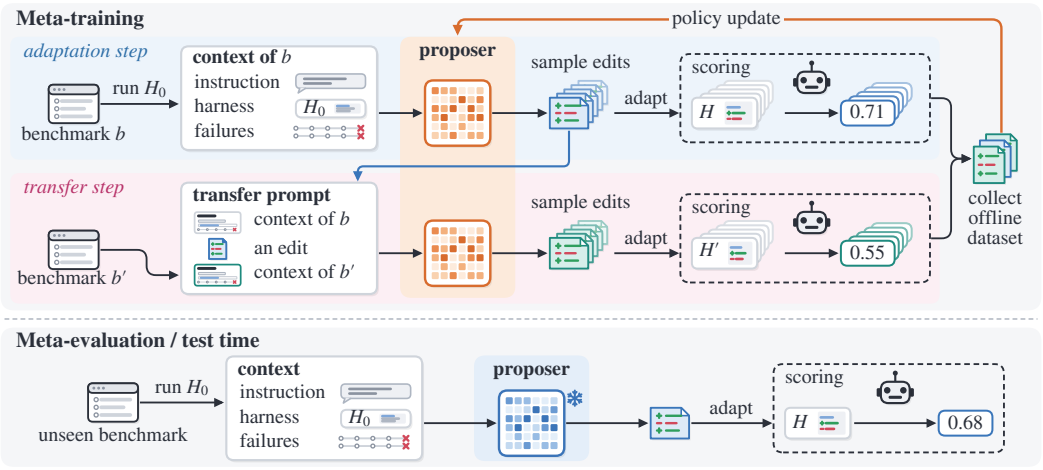


Figure 2: Meta-training and meta-evaluation of LeAH. Top: in the adaptation step, the proposer reads a context of a training benchmark b, which holds the instruction, the initial harness H_0 and failure trajectories under H_0, and samples K edits. In the scoring block, each edit is applied to H_0, the executor runs under each adapted harness $H = H_0 \oplus e$ on the query set of b, and its success rate is the score of that edit. In the transfer step, each edit goes into transfer prompts with its context and a context of another training benchmark b', and the proposer samples edits for b', scored in the same way on b'. The top-ranked edits of both steps, with their prompts, are collected into an offline dataset, on which the policy update fine-tunes the proposer (Algorithm 1). Bottom: at meta-evaluation and test time, the frozen proposer reads one context and writes one edit, which is scored once, with no selection and no weight update.

generalization to other benchmarks part of its objective, adapts from a few failures and is evaluated on agent benchmarks whose tools, interfaces and action spaces differ from those of training.

We refer to Appendix I for related work on reinforcement learning for self-improvement and on meta-learning for optimization.

3 Meta-Learning for Generalizable Harness Adaptation

In this section, we set up harness adaptation as meta-learning and start from ordinary reinforcement learning on the reward of the edit (Section 3.1). We then define generalization across benchmarks as a joint event and take its probability as the objective (Section 3.2), describe learning from an edit as a Bayesian update estimated in context (Section 3.3), derive a practical training loss (Section 3.4) and give the meta-training and meta-evaluation loops (Section 3.5, Figure 2).

3.1 Problem Setup

Problem statement. An agent is a frozen language model, the executor, that acts through a harness: the system prompt it reads and the hook code that runs at fixed points of its agent loop. A benchmark is an environment, with its tools and interface, together with a set of tasks and a grader, and an episode is one run of the executor on one of its tasks. Each benchmark comes with an initial harness, under which the executor fails on some of its tasks. We want a second language model, the proposer, to read a few of these failures and write one edit, new text for the system prompt or new hooks, so that the executor solves more of the other tasks of the benchmark. The proposer should do this on a benchmark it has never seen, without searching on it or updating weights, so it should learn how to generalize.

Problem formulation. Let $b \sim \rho(b)$ denote a benchmark drawn from a population of benchmarks, with initial harness H_0. A context $c \sim q_b(c)$ holds the instruction I to the proposer, H_0 and a support set τ of failure trajectories under H_0. The proposer is a language model with weights θ that reads the context and writes an edit, new text for some fields of the harness, new hooks or both, which

turns H_0 into the edited harness H. With $r_b(H)$ the success rate of the executor under H on the query set of b, a fixed set of tasks disjoint from the support set, the reward of the edit is the success rate of the edited harness:

$$c = \{I, H_0, \tau\}, \qquad e \sim p_\theta(e \mid c), \qquad H = H_0 \oplus e, \qquad r_b(e) = r_b(H).$$

This setup is an instance of meta-learning (Thrun & Pratt, 1998; Finn et al., 2017), with a benchmark as a task, the context holding its support set, and writing the edit as the adaptation step. We keep the support set to a few failures and the adaptation to one edit, since an agent deployed in a new environment cannot be assumed to have many failures to learn from. The adaptation is a single generation rather than a gradient step, as in meta-learners that adapt in context (Santoro et al., 2016; Duan et al., 2016), so the proposer is itself the learned adaptation rule. Benchmarks differ in their tasks, tools and interfaces, so an edit written for one benchmark need not apply to another, and what one proposer can share across benchmarks is this adaptation rule rather than a harness. Our goal is a proposer whose adaptation rule generalizes, so that from a context of a benchmark it never met in training, it writes an edit that improves the executor on that benchmark. The simplest objective trains the proposer with ordinary reinforcement learning on the reward of its edit:

$$\max_\theta \; \mathbb{E}_{b,c}\, \mathbb{E}_{e \sim p_\theta(\cdot|c)} \big[r_b(e) \big], \tag{1}$$

where $\mathbb{E}_{b,c}$ averages over benchmarks $b \sim \rho(b)$ and their contexts $c \sim q_b(c)$. This is the usual meta-learning objective, the performance on the query set after adaptation, averaged over the training benchmarks.

3.2 Generalization as a Joint Objective

Equation (1) asks only that an edit improve its own benchmark. A proposer can raise it by learning edits that suit the training benchmarks, which need not help on a benchmark it has not seen. Meta-learners can likewise fit their training tasks by memorizing them rather than adapting from the support set (Yin et al., 2020).

We therefore define generalization as a joint event: the edit e written for a context c of benchmark b succeeds, and after the proposer learns from (c, e), the edit e' it writes for a context c' of another benchmark $b' \neq b$ also succeeds. Let $p_\theta(e' \mid c'; c, e)$ denote the distribution of edits that the proposer writes for c' after learning from (c, e), defined in Section 3.3. The expected success rate of these edits,

$$T_\theta(c, e) = \mathbb{E}_{b',c'}\, \mathbb{E}_{e' \sim p_\theta(\cdot|c';\, c,e)} \big[r_{b'}(e') \big], \tag{2}$$

is the transfer of (c, e), where $\mathbb{E}_{b',c'}$ averages over $b' \sim \rho(b' \mid b' \neq b)$ and $c' \sim q_{b'}(c')$. Our objective is the probability of the joint event,

$$J(\theta) = \mathbb{E}_{b,c}\, \mathbb{E}_{e \sim p_\theta(\cdot|c)} \big[r_b(e)\, T_\theta(c, e) \big]. \tag{3}$$

Writing out the transfer gives J as an expectation over both edits (Appendix A), in which the proposer writes e for c, then writes e' for c' after learning from (c, e), and $r_b(e)\, r_{b'}(e')$ is the probability that both edits succeed on a query task of their benchmarks. Ordinary reinforcement learning, Equation (1), rewards the first edit alone. Because the proposer writes its own adaptation data, the factor $r_b(e)$ weights what it learns from an edit by the success of that edit on b. As in MLDG (Li et al., 2018), what the proposer learns on b is evaluated on another benchmark b', both drawn from the training benchmarks.

3.3 Learning from an Edit

Learning from (c, e) could mean updating θ on this pair, a form of test-time training (Sun et al., 2020), as in the inner loop of MAML (Finn et al., 2017) or the self-edits of SEAL (Zweiger et al., 2025), but one update for every sampled edit is costly for a language model. We instead define $p_\theta(e' \mid c'; c, e)$ with a hierarchical Bayesian model of the proposer and estimate it in context. Following the hierarchical Bayesian view of meta-learning (Baxter, 1997; Grant et al., 2018), we model the proposer as holding a latent fix strategy z, which describes the failure modes of harnesses and the edits that fix them, with prior $p_\theta(z)$. One z serves both contexts, and each edit depends on z and on its own context, $e \sim p_\theta(e \mid c, z)$ and $e' \sim p_\theta(e' \mid c', z)$. Since z is a property of the

proposer and the contexts are drawn from the benchmarks independently of it, we assume that z is independent of the contexts, $p_\theta(z \mid c, c') = p_\theta(z)$ (Kingma & Welling, 2014; Jiang et al., 2017; Garnelo et al., 2018; Wang et al., 2025a). Learning from (c, e) then changes the belief about z from the prior to the posterior $p_\theta(z \mid c, e) \propto p_\theta(z)\, p_\theta(e \mid c, z)$, and the proposer writes for c' under this posterior:

$$p_\theta(e' \mid c';\, c, e) = \int p_\theta(z \mid c, e)\, p_\theta(e' \mid c', z)\, dz. \tag{4}$$

Equation (4) is a posterior predictive, which in-context learning can estimate without a weight update. Xie et al. (2022) explain in-context learning as implicit Bayesian inference, in which a model infers the latent concept shared by the examples in its prompt and predicts with it. We therefore perform in-context learning over the adaptation example (c, e), placing c, e and then c' in one prompt as in MetaICL (Min et al., 2022), which we call the transfer prompt, and take the edit written for c' as the edit after learning from (c, e):

$$p_\theta(e' \mid c';\, c, e) \approx p_\theta(e' \mid c, e, c'). \tag{5}$$

The transfer prompt is used only in meta-training, and at meta-evaluation the proposer reads one context.

3.4 Deriving a Practical Training Loss

With Equation (5) substituted into the objective, both edits come from the proposer at the same weights, and only their two probabilities depend on θ, so the gradient of J has one term for each edit (Appendix A). The first term trains adaptation examples, with prompt c, completion e and weight $r_b(e)\, T_\theta(c, e)$, in which the transfer is computed through the transfer prompt. The second term trains transfer examples, with prompt (c, e, c'), completion e' and weight $r_b(e)\, r_{b'}(e')$.

We estimate this gradient by Monte Carlo with the proposer at its current weights θ_t, sampling K candidate edits e_j for each of the N contexts c of a round and, for each e_j, K' edits e'_{jk} from transfer prompts with contexts c'_k of benchmarks $b'_k \neq b$. With the measured success rates $\hat{r}$ and the transfer estimate $\hat{T}(c, e_j) = \frac{1}{K'} \sum_{k=1}^{K'} \hat{r}_{b'_k}(e'_{jk})$, this estimate is the negative gradient of a training loss $\mathcal{L}$:

$$\mathcal{L}(\theta) = -\frac{1}{NK} \sum_{c} \sum_{j=1}^{K} \hat{r}_b(e_j) \Big[\hat{T}(c, e_j) \log p_\theta(e_j \mid c) + \frac{1}{K'} \sum_{k=1}^{K'} \hat{r}_{b'_k}(e'_{jk}) \log p_\theta(e'_{jk} \mid c, e_j, c'_k) \Big], \tag{6}$$

in which the measured weights are constants, so that $\mathcal{L}$ is a weighted supervised fine-tuning loss on adaptation and transfer examples. Ordinary reinforcement learning keeps only the adaptation examples, weighted by $\hat{r}_b(e_j)$ alone, which gives the gradient of Equation (1) in expectation.

In practice, as a surrogate for the weights of Equation (6), we standardize the reward and the transfer of each candidate over the candidates of its context and rank the candidates by the sum of the two (Appendix A). We add the two rather than multiply them, since both are measured as gains over H_0 and can be negative. Ordinary reinforcement learning ranks by the adaptation reward alone. We found ReST (Gulcehre et al., 2023; Singh et al., 2024) more stable to train than GRPO (Shao et al., 2024) in this setting (Appendix F), as SEAL also found (Zweiger et al., 2025), so for both objectives we keep the top-ranked candidates of each context as adaptation examples. Under the joint objective, the highest-ranked of these also add, for each other training benchmark, their transfer edit with the highest success rate as a transfer example (Appendix B). The loss is the standard supervised fine-tuning loss on the proposer's reply, with the prompt masked.

3.5 Meta-Training and Meta-Evaluation

The tasks of each training benchmark are split into a meta-train split and a meta-test split, each with its own query set and support pool. Each round of meta-training (Figure 2 and Algorithm 1) draws contexts from the meta-train splits, runs the adaptation and transfer steps with the proposer at θ_t and trains the proposer with SFT on the selected examples $\mathcal{D}$ (Section 3.4) to obtain θ_{t+1}. The contexts c' of the transfer step come from the other training benchmarks, so each training benchmark serves as b in one draw and as b' in another. Meta-evaluation keeps only the adaptation

Algorithm 1 Learning to Adapt Harnesses (LeAH)

Input: SFT proposer θ_0, number of rounds R, numbers M and M' of kept candidates

for $t = 0, \ldots, R-1$ **do**
  $\mathcal{D} \leftarrow \emptyset$
  for each context c of a training benchmark b **do**
    Sample K edits $e_j \sim p_{\theta_t}(\cdot \mid c)$ ▷ adaptation step
    Measure $\hat{r}_b(e_j)$ of each edit on the query set of b
    Draw contexts $c'_1, \ldots, c'_{K'}$ of training benchmarks $b'_k \neq b$
    for each e_j and $k = 1, \ldots, K'$ **do**
      Sample $e'_{jk} \sim p_{\theta_t}(\cdot \mid c, e_j, c'_k)$ ▷ transfer step
      Measure $\hat{r}_{b'_k}(e'_{jk})$ on the query set of b'_k
    $\hat{T}(c, e_j) \leftarrow \frac{1}{K'} \sum_k \hat{r}_{b'_k}(e'_{jk})$ for each e_j
  $s(c, e_j) \leftarrow \tilde{r}_b(e_j) + \tilde{T}(c, e_j)$, each standardized over the K edits
  $s'(e'_{jk}) \leftarrow \tilde{r}_b(e_j) + \tilde{r}_{b'_k}(e'_{jk})$
  $\mathcal{D} \leftarrow \mathcal{D} \cup \{(c, e_j) : j \in \mathrm{top}_M(s)\}$
  $\mathcal{D} \leftarrow \mathcal{D} \cup \{((c, e_j, c'_k), e'_{jk}) : j \in \mathrm{top}_{M'}(s)\}$, the k of highest s' per b'
  $\theta_{t+1} \leftarrow \mathrm{SFT}(\theta_t, \mathcal{D})$

Meta-evaluation and deployment

Given a context c of a new benchmark, sample one edit $e \sim p_{\theta_R}(\cdot \mid c)$
Run the executor under $H_0 \oplus e$, with no selection and no weight update

step, as at deployment, and we report the benchmark's metric for the executor under the edited harness on the query set. We evaluate on held-out tasks of the training benchmarks, the query sets of their meta-test splits, and on unseen benchmarks, which never enter training and are out of distribution for the proposer.

4 Experimental Setup

In this section, we evaluate whether a meta-trained proposer can adapt a harness from a few failures, on its training benchmarks and on benchmarks it has never seen. We ask **(i)** whether, given a few failures of a training benchmark, the proposer writes an edit that improves the executor on disjoint query tasks of that benchmark; **(ii)** whether this ability generalizes to unseen benchmarks, whose tasks, tools and interfaces the proposer never met in training; **(iii)** whether this generalization comes from the joint objective rather than from ordinary reinforcement learning on the reward of the edit; and **(iv)** what failure modes and fixes the proposer learns, and how they change under the joint objective.

Benchmarks. Every benchmark runs on its official environment and grader.

- *Training benchmarks:* household tasks in ALFWorld (Shridhar et al., 2021), database queries through SQL tool calls in DBBench (Liu et al., 2024) and web shopping in WebShop (Yao et al., 2022).
- *Unseen benchmarks:* the text environments ScienceWorld (Wang et al., 2022), BabyAI (Chevalier-Boisvert et al., 2019; Xi et al., 2025) and TextWorld (Côté et al., 2018), the Gridworld environment of LLF-Bench (Cheng et al., 2023), multi-hop tool use in ToolHop (Ye et al., 2025) and text-to-SQL in BIRD (Li et al., 2023).

A harness is the executor's system prompt together with hook functions that run at fixed points of its agent loop, and an edit may rewrite the prompt and add or change hooks. The proposer's prompt is the same for every benchmark except for the harness, the failures and one sentence on how the executor's actions reach the hooks (Appendix D). The initial harness H_0 of each benchmark is its default agent runtime, as in Harness-R1 (Shao et al., 2026). Each training benchmark is split in half into a meta-train and a meta-test split, each with a query set of 50 tasks and a support pool of the executor's failures under H_0 on its other tasks, and each unseen benchmark has one query

set and one support pool. A context holds five failure trajectories from a support pool. We refer to Appendix B for details on the benchmarks.

Models and training. The executor is Qwen3.5-9B (Qwen Team, 2026a), frozen throughout. Following prior harness proposers, which start from SFT on a teacher's outputs (Shao et al., 2026; Zhang et al., 2026b;a), the proposer starts from the same model after SFT on edits that a teacher model, DeepSeek-V4.1-Flash (DeepSeek-AI, 2026), writes for contexts of the training benchmarks. Meta-training runs three rounds of the loop in Section 3.5 with low-rank adaptation (LoRA) (Hu et al., 2022), selecting and training on examples as in Section 3.4. The proposer samples $K = 30$ candidate edits per context in the first round and $K = 10$ in the next two, and $K' = 6$ transfer edits per candidate, three for each of the two other training benchmarks. Appendix B lists the settings and hyperparameters, and Appendix D gives the prompts and how failure trajectories are compiled. We refer to Appendix E for additional results with different proposer models and for the transfer of the learned proposer to different executors.

Evaluation. For each evaluation benchmark, we draw three contexts at random, from the meta-test split of a training benchmark or the support pool of an unseen benchmark, and sample three edits for each at the training temperature. An illegal edit falls back to H_0. Each meta-trained proposer comes from one meta-training run. We report the benchmark's own metric on the 50 query tasks, the success rate or, for ScienceWorld, BabyAI and TextWorld, the score, as the mean over these edits with its standard error (Appendix B).

Baselines. Every method writes edits from the same contexts and is compared with H_0 on the same query tasks. Each adaptation costs one generation of the proposer, so we do not compare with harness search, which runs hundreds to thousands of task executions on each new benchmark (Lee et al., 2026; Lin et al., 2026; Wang et al., 2026b) or searches over the agent's traces at test time (Nie et al., 2026).

- *No proposer:* H_0 with the five failure trajectories of the context appended to its prompt.
- *Base model and SFT*: Qwen3.5-9B without training, and the proposer after SFT and before meta-training. The base model is the same model as the executor, so it is also a Reflexion-style baseline (Shinn et al., 2023), which reflects on its own failures and writes what it learns into the executor's harness, here for new tasks rather than for another attempt at the failed ones.
- *Teacher:* DeepSeek-V4.1-Flash (DeepSeek-V4.1 in Table 1), which tests whether meta-training goes beyond the model whose edits the proposer was fine-tuned on.
- *Harness-R1* (Shao et al., 2026), a learned proposer trained on the same three benchmarks, whose code hooks at four fixed points of the agent loop read a runtime context built for each of them, with state fields and predicates such as an unselected required option in WebShop. This context exists only for ALFWorld, DBBench and WebShop, so we run the released checkpoint with its own method on the training benchmarks only, and the same checkpoint with our prompt on all benchmarks.
- *Ordinary reinforcement learning*, the matched comparison for question (iii), is the same training without transfer examples and differs from LeAH only in the objective. It follows the recipe of Harness Learning (Zhang et al., 2026a): SFT on teacher edits, then reinforcement learning on the reward of the edit, here with the selection and loss of Section 3.4.

5 Results and Discussion

5.1 Adaptation on Training Benchmarks

We first ask whether the meta-trained proposer, given a few failures of a training benchmark, writes an edit that improves the executor on disjoint query tasks of that benchmark. Table 1 shows that LeAH raises the average success rate on the training benchmarks from 35.3% under H_0 to 48.5%, improves on each of the three training benchmarks and is higher than every other 9B proposer and than its teacher (42.7%).

The baselines show that this ability comes neither with the model nor from imitation alone. Given the same contexts, the base Qwen3.5-9B writes edits that lower the average on the training benchmarks to 32.4%, below the 35.3% of H_0. SFT raises it to 43.2%, against 42.7% for its teacher,

Table 1: Adaptation on held-out tasks of the training benchmarks and on unseen benchmarks. Cells are means over nine edits (three random contexts, three edits each) with standard errors, in success rate (%) or, marked †, in score from 0 to 100. SciWorld is ScienceWorld. H_0 + failures appends the five failures of a context to the prompt of H_0. Harness-R1 uses its own runtime context, which exists only for the training benchmarks, and Harness-R1* is the same checkpoint with our prompt. LeAH w/o SFT is meta-trained from the base model, without SFT. Bold and underline mark the best and second-best 9B proposer.

	Training benchmarks (held-out tasks)				Unseen benchmarks						
Proposer	WebShop	DBBench	ALFWorld	Avg.	Gridworld	BIRD	BabyAI†	SciWorld†	TextWorld†	ToolHop	Avg.
H_0	12.0	42.0	52.0	35.3	74.0	54.0	73.6	22.6	38.2	46.0	51.4
No proposer											
H_0 + failures	11.8$_{\pm 0.6}$	49.1$_{\pm 1.1}$	50.7$_{\pm 1.3}$	37.2$_{\pm 0.6}$	53.6$_{\pm 1.6}$	45.1$_{\pm 0.8}$	62.5$_{\pm 1.5}$	38.6$_{\pm 2.6}$	31.9$_{\pm 3.5}$	47.6$_{\pm 0.6}$	46.5$_{\pm 0.8}$
Teacher											
DeepSeek-V4.1	18.4$_{\pm 2.9}$	52.9$_{\pm 1.6}$	56.9$_{\pm 2.5}$	42.7$_{\pm 1.4}$	87.6$_{\pm 3.3}$	39.1$_{\pm 7.6}$	81.3$_{\pm 1.7}$	34.0$_{\pm 3.6}$	52.1$_{\pm 2.7}$	52.7$_{\pm 0.6}$	57.8$_{\pm 1.6}$
9B proposers											
Qwen3.5-9B	9.8$_{\pm 1.3}$	50.9$_{\pm 2.5}$	36.7$_{\pm 3.5}$	32.4$_{\pm 1.5}$	86.4$_{\pm 3.0}$	53.3$_{\pm 1.2}$	69.6$_{\pm 3.8}$	24.4$_{\pm 1.1}$	25.2$_{\pm 5.9}$	**49.6**$_{\pm 1.1}$	51.4$_{\pm 1.3}$
LeAH w/o SFT	7.8$_{\pm 2.0}$	50.7$_{\pm 1.2}$	45.1$_{\pm 5.8}$	34.5$_{\pm 2.1}$	75.8$_{\pm 9.7}$	53.3$_{\pm 1.3}$	73.7$_{\pm 1.6}$	25.3$_{\pm 1.6}$	28.1$_{\pm 5.0}$	40.7$_{\pm 6.6}$	49.5$_{\pm 2.2}$
Harness-R1	22.7$_{\pm 0.9}$	54.9$_{\pm 1.5}$	47.6$_{\pm 3.6}$	41.7$_{\pm 1.3}$	–	–	–	–	–	–	–
Harness-R1*	9.3$_{\pm 2.4}$	47.3$_{\pm 1.4}$	52.2$_{\pm 1.7}$	36.3$_{\pm 1.1}$	67.1$_{\pm 8.9}$	53.1$_{\pm 0.5}$	73.4$_{\pm 0.2}$	20.4$_{\pm 2.2}$	32.5$_{\pm 3.7}$	46.2$_{\pm 0.5}$	48.8$_{\pm 1.6}$
SFT	21.1$_{\pm 3.8}$	52.9$_{\pm 1.5}$	55.6$_{\pm 3.5}$	43.2$_{\pm 1.8}$	80.4$_{\pm 9.6}$	39.1$_{\pm 6.9}$	74.8$_{\pm 2.5}$	26.2$_{\pm 2.8}$	29.8$_{\pm 6.6}$	46.0$_{\pm 5.0}$	49.4$_{\pm 2.5}$
LeAH (ours)	**29.8**$_{\pm 1.3}$	**55.3**$_{\pm 2.2}$	**60.4**$_{\pm 3.8}$	**48.5**$_{\pm 1.5}$	**90.9**$_{\pm 2.2}$	**54.7**$_{\pm 0.7}$	**78.5**$_{\pm 1.8}$	**32.8**$_{\pm 3.0}$	**42.0**$_{\pm 2.1}$	48.4$_{\pm 1.3}$	**57.9**$_{\pm 0.8}$

which is what imitating the teacher's edits can provide. Meta-training then adds the rest of the gain, which suggests that the proposer learns which edits help on tasks beyond the observed failures, rather than only how a stronger model writes edits. Meta-training from the base model instead (LeAH w/o SFT), with the same rounds, contexts, candidates and objective, raises this average only from 32.4% to 34.5% and stays below H_0 on the unseen benchmarks (49.5), so meta-training builds on SFT. We find that without SFT, the proposer often struggles to write an edit that follows the instruction.

Without a proposer, the same five failures barely improve the executor on the query tasks. Appending them to H_0 gives an average of 37.2% on the training benchmarks, below H_0 on WebShop and ALFWorld, while LeAH's proposer is trained to turn such failures into an edit that helps on query tasks disjoint from them.

Harness-R1 improves over H_0 (41.7% against 35.3%) with its own runtime context, whose hooks read state and predicates built for these three benchmarks, but the same checkpoint writing free-form edits from our prompt keeps little of this gain (36.3%) and falls below H_0 on WebShop. LeAH writes free-form edits from the same input and is higher on average than Harness-R1 in either setting, without state or predicates designed for each benchmark.

5.2 Generalization to Unseen Benchmarks

We then ask whether this ability generalizes to benchmarks whose tasks, tools and interfaces the proposer never met in training. On the six unseen benchmarks, LeAH raises the average from 51.4 under H_0 to 57.9 (Table 1), improves over H_0 on five of them and is comparable to H_0 on BIRD. This average matches that of the teacher (57.8), while every other 9B proposer in Table 1 stays at or below H_0.

The SFT proposer indicates that this generalization is not inherited from imitation. Trained on the teacher's edits for the training benchmarks, it matches the teacher on the training benchmarks but falls below H_0 on the unseen benchmarks (49.4), with the largest losses on BIRD and TextWorld. Meta-training from the same SFT proposer removes the loss on BIRD and turns the loss on TextWorld into a gain over H_0. This suggests that imitation teaches the form of an edit, while an adaptation rule that holds on a new interface is learned from the outcomes of the proposer's own edits.

Table 2: Ordinary reinforcement learning against the joint objective, from the same SFT proposer with the same adaptation steps. Cells as in Table 1. Bold and underline mark the best and second-best proposer.

	Training benchmarks (held-out tasks)				Unseen benchmarks						
Proposer	WebShop	DBBench	ALFWorld	Avg.	Gridworld	BIRD	BabyAI†	SciWorld†	TextWorld†	ToolHop	Avg.
H_0	12.0	42.0	52.0	35.3	74.0	54.0	73.6	22.6	38.2	46.0	51.4
SFT	21.1$_{\pm 3.8}$	52.9$_{\pm 1.5}$	55.6$_{\pm 3.5}$	43.2$_{\pm 1.8}$	<u>80.4</u>$_{\pm 9.6}$	39.1$_{\pm 6.9}$	74.8$_{\pm 2.5}$	26.2$_{\pm 2.8}$	29.8$_{\pm 6.6}$	46.0$_{\pm 5.0}$	49.4$_{\pm 2.5}$
Ordinary RL	<u>25.1</u>$_{\pm 2.5}$	<u>55.1</u>$_{\pm 1.7}$	**62.7**$_{\pm 1.6}$	<u>47.6</u>$_{\pm 1.1}$	78.9$_{\pm 9.7}$	<u>47.8</u>$_{\pm 6.0}$	**78.6**$_{\pm 0.9}$	<u>26.5</u>$_{\pm 2.4}$	<u>39.0</u>$_{\pm 1.8}$	**50.4**$_{\pm 0.9}$	<u>53.5</u>$_{\pm 2.0}$
Joint (LeAH)	**29.8**$_{\pm 1.3}$	**55.3**$_{\pm 2.2}$	<u>60.4</u>$_{\pm 3.8}$	**48.5**$_{\pm 1.5}$	**90.9**$_{\pm 2.2}$	**54.7**$_{\pm 0.7}$	<u>78.5</u>$_{\pm 1.8}$	**32.8**$_{\pm 3.0}$	**42.0**$_{\pm 2.1}$	<u>48.4</u>$_{\pm 1.3}$	**57.9**$_{\pm 0.8}$

Harness-R1 has its own runtime context for the three training benchmarks, and on the unseen benchmarks, which have none, we run it with our prompt. It stays below H_0 on average (48.8), so its gains depend on the runtime context built for its benchmarks. Trained on the same three benchmarks without such context, LeAH learns an adaptation rule that carries over to new interfaces.

Without a proposer, appending the failures to H_0 lowers the average on the unseen benchmarks to 46.5. It falls below H_0 on four of the six benchmarks and is higher than LeAH only on ScienceWorld, so showing the executor its failures helps on few new benchmarks, whereas the adaptation rule that LeAH learns from the outcomes of its edits holds across them.

5.3 Does the Joint Objective Drive Generalization?

We next ask whether the gain on the unseen benchmarks comes from the joint objective. Ordinary reinforcement learning starts from the same SFT proposer and runs the same adaptation steps, but optimizes Equation (1) instead of Equation (3). Table 2 shows that the two objectives reach comparable success on the training benchmarks (47.6% and 48.5%) but differ on the unseen benchmarks. Measured from H_0, ordinary reinforcement learning gains about a third as much as LeAH on the average over the unseen benchmarks (53.5 against 57.9, from 51.4).

LeAH is higher on four of the six unseen benchmarks by more than a standard error. Ordinary reinforcement learning rewards an edit that succeeds on its own benchmark, while the joint objective also rewards it for teaching the proposer to write a successful edit for another benchmark. We interpret the gain on the unseen benchmarks as the effect of this objective, and Section 5.4 examines what the edits of the two objectives contain. We refer to Appendix G for LeAH trained on no more examples than ordinary reinforcement learning and for contexts with fewer failures.

5.4 What the Proposer Learns

Finally, we ask what failure modes and fixes the proposer learns, and how they change under the joint objective. We read the final edits of the SFT proposer, ordinary reinforcement learning and LeAH on the six unseen benchmarks, with the failures each edit was written from and the episodes it produced (Appendix H). LeAH more often writes its fix from the new benchmark. On the five benchmarks with a rule of their own, 17 of LeAH's 45 edits use a rule specific to the new benchmark, against 7 of ordinary reinforcement learning's and 4 of the SFT proposer's (Figure 3c).

A ScienceWorld task is completed only if the executor focuses on the object that the task names, and 43 of its 50 query tasks ask for this first, as in "First, focus on the aluminum foil." LeAH's edits more often state this step in the terms of the task, while three of the nine edits of ordinary reinforcement learning leave the focus to the end of the task. In Figure 3a, the executor under LeAH's edit reasons "The task requires me to first focus on the aluminum foil" and focuses on it (score 60), while under the edit of ordinary reinforcement learning it picks up the foil and never focuses (score 14). Over the 450 episodes of each proposer, the first focus is on the right object in 46% of LeAH's episodes and 37% of those of ordinary reinforcement learning, against 14 of the 50 episodes (28%) under H_0 (Appendix H.2).

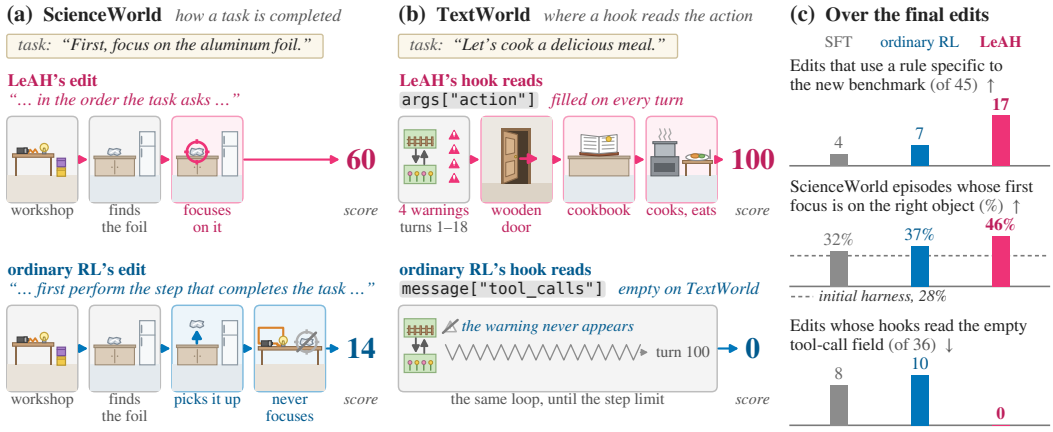


Figure 3: Edits of LeAH and ordinary reinforcement learning, each pair written from the same context, on two unseen benchmarks. (a) ScienceWorld: the phrase of each edit that decides the episode and what the executor then does. (b) TextWorld: where each edit's hook reads the action and whether its warning appears. (c) Over the final edits of each proposer for the unseen benchmarks, with the initial harness dashed.

On ScienceWorld, BabyAI, TextWorld and Gridworld, the executor writes its action as plain text, and the harness passes it to the hooks as an argument, while the tool-call field that the training benchmarks fill stays empty. In Figure 3b, the two edits for the same TextWorld context both add a hook that warns the executor when it repeats a cycle of actions. LeAH's hook reads the action from its argument, so its warning appears and the executor leaves the loop and cooks and eats the meal (score 100), while the hook of ordinary reinforcement learning reads the empty field, so its warning never appears and the executor walks between two rooms until the step limit (score 0). None of LeAH's 36 edits for these four benchmarks reads the tool-call field, against 10 of ordinary reinforcement learning's and 8 of the SFT proposer's (Figure 3c and Appendix H.3).

Both differences follow from the objective. Ordinary reinforcement learning rewards an edit that succeeds on the benchmark it was written for, where a general rule works and the tool-call field is filled. The joint objective also rewards it for teaching the proposer to write a successful edit for another benchmark. A transfer edit that copies the system prompt of its source solves fewer tasks of the target, and the share of transfer edits that carry the interface of their source falls over the rounds (Appendix H.4). Ordinary reinforcement learning therefore learns what a successful edit looks like on its training benchmarks, while LeAH learns the adaptation rule of Section 3.1.

6 Conclusion

We asked whether a proposer meta-trained across agent benchmarks can adapt an executor's harness to a benchmark from a few failures, including benchmarks it has never seen. To learn an adaptation rule that generalizes, LeAH trains the proposer with a joint objective, under which an edit should succeed on its own benchmark and the edit that the proposer writes for another benchmark after learning from it should also succeed. Estimating this learning in context turns the objective into a weighted supervised loss on adaptation and transfer examples. Meta-trained on three agent benchmarks, the 9B proposer improves the executor on held-out tasks of these benchmarks and on six unseen benchmarks, where it matches its teacher. Training without transfer examples reaches comparable success on the training benchmarks but gains about a third as much on the unseen benchmarks. On a new benchmark, the edits of LeAH more often use the rules of that benchmark and read the executor's action where that benchmark passes it, while those of ordinary reinforcement learning more often carry over the forms that worked on its training benchmarks.

Limitations

LeAH adapts the harness of each benchmark independently, and the proposer does not change between adaptations. We do not study how the proposer could internalize the harnesses it adapts (Wang et al., 2026c), so that each adaptation improves the next and the proposer keeps getting stronger as it adapts. Each adaptation is also a single edit written from failures under H_0, and we do not study how the proposer could revise the edited harness over further rounds, from the failures that remain under it (Zhang et al., 2026a). We leave both for future work.

Impact Statement

This paper presents work whose goal is to advance the field of machine learning. The proposer writes code hooks that run inside an agent's loop. In our experiments, a static audit limits their imports and blocks file, network and process access (Appendix D), and proposer-written code deployed in real environments would need similar sandboxing.

References

Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *International Conference on Learning Representations*, 2026. URL https://arxiv.org/abs/2507.19457.

Marcin Andrychowicz, Misha Denil, Sergio Gómez Colmenarejo, Matthew W. Hoffman, David Pfau, Tom Schaul, Brendan Shillingford, and Nando de Freitas. Learning to learn by gradient descent by gradient descent. In *Advances in Neural Information Processing Systems*, volume 29, 2016. URL https://proceedings.neurips.cc/paper_files/paper/2016/hash/fb87582825f9d28a8d42c5e5e5e8b23d-Abstract.html.

Jonathan Baxter. A Bayesian/information theoretic model of learning to learn via multiple task sampling. *Machine Learning*, 28(1):7–39, 1997. doi: 10.1023/A:1007327622663.

Daniel A. Braun, Ad Aertsen, Daniel M. Wolpert, and Carsten Mehring. Motor task variation induces structural learning. *Current Biology*, 19(4):352–357, February 2009. doi: 10.1016/j.cub.2009.01.036. URL https://doi.org/10.1016/j.cub.2009.01.036.

Tingyang Chen, Shuo Lu, Kang Zhao, Weicheng Meng, Hanlin Teng, Tianhao Li, Chao Li, Xule Liu, Jian Liang, Zhizhong Zhang, Yuan Xie, Heng Qu, Kun Shao, and Jian Luan. HarnessX: A composable, adaptive, and evolvable agent harness foundry, 2026. URL https://arxiv.org/abs/2606.14249. arXiv preprint, version 3, 23 July 2026.

Wei-Yu Chen, Yen-Cheng Liu, Zsolt Kira, Yu-Chiang Frank Wang, and Jia-Bin Huang. A closer look at few-shot classification. In *International Conference on Learning Representations*, 2019. URL https://arxiv.org/abs/1904.04232.

Ching-An Cheng, Andrey Kolobov, Dipendra Misra, Allen Nie, and Adith Swaminathan. LLF-Bench: Benchmark for interactive learning from language feedback, 2023. URL https://arxiv.org/abs/2312.06853.

Maxime Chevalier-Boisvert, Dzmitry Bahdanau, Salem Lahlou, Lucas Willems, Chitwan Saharia, Thien Huu Nguyen, and Yoshua Bengio. BabyAI: A platform to study the sample efficiency of grounded language learning. In *International Conference on Learning Representations*, 2019.

Anne G. E. Collins and Michael J. Frank. Cognitive control over learning: Creating, clustering, and generalizing task-set structure. *Psychological Review*, 120(1):190–229, January 2013. doi: 10.1037/a0030852. URL https://doi.org/10.1037/a0030852.

Marc-Alexandre Côté, Ákos Kádár, Xingdi Yuan, et al. TextWorld: A learning environment for text-based games. In *Computer Games Workshop at IJCAI 2018*, 2018.

DeepSeek-AI. DeepSeek-V4.1-Flash: Pushing the limits of KV cache compression, September 2026. URL https://arxiv.org/abs/2609.19969. arXiv preprint arXiv:2609.19969.

Yan Duan, John Schulman, Xi Chen, Peter L. Bartlett, Ilya Sutskever, and Pieter Abbeel. RL^2: Fast reinforcement learning via slow reinforcement learning, 2016. URL https://arxiv.org/abs/1611.02779v2. Version 2, 10 November 2016.

Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pp. 1126–1135. PMLR, 2017. URL https://proceedings.mlr.press/v70/finn17a.html.

Marta Garnelo, Jonathan Schwarz, Dan Rosenbaum, Fabio Viola, Danilo J. Rezende, S. M. Ali Eslami, and Yee Whye Teh. Neural processes, 2018. URL https://arxiv.org/abs/1807.01622. arXiv preprint arXiv:1807.01622.

Erin Grant, Chelsea Finn, Sergey Levine, Trevor Darrell, and Thomas Griffiths. Recasting gradient-based meta-learning as hierarchical Bayes. In *International Conference on Learning Representations*, 2018. URL https://arxiv.org/abs/1801.08930.

Caglar Gulcehre, Tom Le Paine, Srivatsan Srinivasan, Ksenia Konyushkova, Lotte Weerts, Abhishek Sharma, Aditya Siddhant, Alex Ahern, Miaosen Wang, Chenjie Gu, Wolfgang Macherey, Arnaud Doucet, Orhan Firat, and Nando de Freitas. Reinforced self-training (ReST) for language modeling, 2023. URL https://arxiv.org/abs/2308.08998. arXiv preprint arXiv:2308.08998.

Yunhui Guo, Noel C. Codella, Leonid Karlinsky, James V. Codella, John R. Smith, Kate Saenko, Tajana Rosing, and Rogerio Feris. A broader study of cross-domain few-shot learning. In *European Conference on Computer Vision*, 2020. URL https://arxiv.org/abs/1912.07200.

Harry F. Harlow. The formation of learning sets. *Psychological Review*, 56(1):51–65, 1949.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

Nathan Hu, Eric Mitchell, Christopher D. Manning, and Chelsea Finn. Meta-learning online adaptation of language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 4418–4432. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.emnlp-main.268. URL https://aclanthology.org/2023.emnlp-main.268/.

Yulun Jiang, Liangze Jiang, Damien Teney, Michael Moor, and Maria Brbić. Meta-RL induces exploration in language agents. In *International Conference on Learning Representations*, 2026. URL https://arxiv.org/abs/2512.16848.

Zhuxi Jiang, Yin Zheng, Huachun Tan, Bangsheng Tang, and Hanning Zhou. Variational deep embedding: An unsupervised and generative approach to clustering. In *International Joint Conference on Artificial Intelligence*, 2017. URL https://arxiv.org/abs/1611.05148.

Charles Kemp, Amy Perfors, and Joshua B. Tenenbaum. Learning overhypotheses with hierarchical Bayesian models. *Developmental Science*, 10(3):307–321, 2007.

Diederik P. Kingma and Max Welling. Auto-encoding variational Bayes. In *International Conference on Learning Representations*, 2014. URL https://arxiv.org/abs/1312.6114.

Harrison Lee, Samrat Phatale, Hassan Mansoor, Thomas Mesnard, Johan Ferret, Kellie Ren Lu, Colton Bishop, Ethan Hall, Victor Carbune, Abhinav Rastogi, and Sushant Prakash. RLAIF vs. RLHF: Scaling reinforcement learning from human feedback with AI feedback. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 26874–26901. PMLR, 2024. URL https://proceedings.mlr.press/v235/lee24t.html.

Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-Harness: End-to-end optimization of model harnesses. In *Conference on Language Modeling (COLM)*, 2026. URL https://arxiv.org/abs/2603.28052.

Da Li, Yongxin Yang, Yi-Zhe Song, and Timothy M. Hospedales. Learning to generalize: Meta-learning for domain generalization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018. doi: 10.1609/aaai.v32i1.11596. URL https://doi.org/10.1609/aaai.v32i1.11596.

Jinyang Li, Binyuan Hui, Ge Qu, et al. Can LLM already serve as a database interface? A BIg bench for large-scale database grounded text-to-SQLs. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2023.

Ke Li and Jitendra Malik. Learning to optimize. In *International Conference on Learning Representations*, 2017. URL https://arxiv.org/abs/1606.01885.

Zhenguo Li, Fengwei Zhou, Fei Chen, and Hang Li. Meta-SGD: Learning to learn quickly for few-shot learning, 2017. URL https://arxiv.org/abs/1707.09835.

Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shihan Dou, Zhiheng Xi, Xuanjing Huang, Hang Yan, Zhenhua Han, Tao Gui, and Yu-Gang Jiang. Agentic Harness Engineering: Observability-driven automatic evolution of coding-agent harnesses, 2026. URL https://arxiv.org/abs/2604.25850. arXiv preprint, version 4, 18 May 2026.

Alexander H. Liu, Kartik Khandelwal, Sandeep Subramanian, et al. Ministral 3, January 2026. URL https://arxiv.org/abs/2601.08584.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=zAdUB0aCTQ.

Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding R1-Zero-like training: A critical perspective. In *Conference on Language Modeling (COLM)*, 2025. URL https://arxiv.org/abs/2503.20783.

Yingzi Ma, Yulong Cao, Nanxi Li, Fang Wu, Yizhe Zhang, James Zou, Yejin Choi, and Chaowei Xiao. HarnessTTT: Learn to optimize harness at test-time for scientific discovery, October 2026. URL https://arxiv.org/abs/2610.17370. Version 1, October 12, 2026.

Luke Metz, James Harrison, C. Daniel Freeman, Amil Merchant, Lucas Beyer, James Bradbury, Naman Agrawal, Ben Poole, Igor Mordatch, Adam Roberts, and Jascha Sohl-Dickstein. VeLO: Training versatile learned optimizers by scaling up, 2022. URL https://arxiv.org/abs/2211.09760. arXiv preprint arXiv:2211.09760.

Sewon Min, Mike Lewis, Luke Zettlemoyer, and Hannaneh Hajishirzi. MetaICL: Learning to learn in context. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 2791–2809, 2022. URL https://aclanthology.org/2022.naacl-main.201/.

Jun Nie, Yonggang Zhang, Jun Song, Qianshu Cai, Dahai Yu, Yike Guo, Xinmei Tian, and Bo Han. TTHE: Test-time harness evolution, 2026. URL https://arxiv.org/abs/2607.08124. arXiv preprint arXiv:2607.08124.

OpenAI. gpt-oss-120b & gpt-oss-20b model card, August 2025. URL https://arxiv.org/abs/2508.10925.

Jing-Cheng Pang, Pengyuan Wang, Kaiyuan Li, Xiong-Hui Chen, Jiacheng Xu, Zongzhang Zhang, and Yang Yu. Language model self-improvement by reinforcement learning contemplation. In *International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=38E4yUbrgr.

Sungho Park, Wonjoong Kim, Rongyuan Tan, Jue Zhang, Wook-Shin Han, Pengfei Gao, Chanyoung Park, Yongqiang Yao, Rao Fu, Elsie Nallipogu, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. AutoSaddler: Automatic harness optimization with durable updates from agent execution traces, August 2026. URL https://arxiv.org/abs/2608.23041. Version 1, 24 August 2026.

Archiki Prasad, Weizhe Yuan, Richard Yuanzhe Pang, Jing Xu, Maryam Fazel-Zarandi, Mohit Bansal, Sainbayar Sukhbaatar, Jason E. Weston, and Jane Yu. Self-consistency preference optimization. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 49737–49751. PMLR, 2025. URL https://proceedings.mlr.press/v267/prasad25a.html.

Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chenguang Zhu, and Michael Zeng. Automatic prompt optimization with "gradient descent" and beam search. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. URL https://arxiv.org/abs/2305.03495.

Yuxiao Qu, Matthew Y. R. Yang, Amrith Setlur, Lewis Tunstall, Edward Emanuel Beeching, Ruslan Salakhutdinov, and Aviral Kumar. Optimizing test-time compute via meta reinforcement finetuning. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 50893–50925. PMLR, 2025. URL https://proceedings.mlr.press/v267/qu25g.html.

Qwen Team. Qwen3.5: Towards native multimodal agents, February 2026a. URL https://qwen.ai/blog?id=qwen3.5. Blog post.

Qwen Team. Qwen3.8-Max: A new bar for coding and cowork, August 2026b. URL https://qwen.ai/blog?id=qwen3.8. Blog post.

Som Sagar, Shasha Li, Hejie Cui, Ransalu Senanayake, and Sercan Ö. Arık. Dynamic harness search: Building multi-agent systems per-query via prediction, October 2026. URL https://arxiv.org/abs/2610.04137. Version 1, October 2, 2026.

Adam Santoro, Sergey Bartunov, Matthew Botvinick, Daan Wierstra, and Timothy Lillicrap. Meta-learning with memory-augmented neural networks. In *Proceedings of the 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pp. 1842–1850. PMLR, 2016. URL https://proceedings.mlr.press/v48/santoro16.html.

Shuai Shao, Kangning Zhang, Qingyao Li, Shijian Wang, Hao Wang, Wenxiang Jiao, Yuan Lu, Yi Guo, Weiwen Liu, and Weinan Zhang. Harness-R1: Learning to edit executable runtime harnesses from agent failure trajectories, August 2026. URL https://arxiv.org/abs/2608.02276. Version 1, August 3, 2026.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models, 2024. URL https://arxiv.org/abs/2402.03300.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html.

Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. ALFWorld: Aligning text and embodied environments for interactive learning. In *International Conference on Learning Representations*, 2021.

Avi Singh, John D. Co-Reyes, Rishabh Agarwal, Ankesh Anand, Piyush Patil, Xavier Garcia, Peter J. Liu, James Harrison, Jaehoon Lee, Kelvin Xu, Aaron Parisi, Abhishek Kumar, Alex Alemi, Alex Rizkowsky, Azade Nova, Ben Adlam, Bernd Bohnet, Gamaleldin Elsayed, Hanie Sedghi, Igor Mordatch, Isabelle Simpson, Izzeddin Gur, Jasper Snoek, Jeffrey Pennington, Jiri Hron, Kathleen Kenealy, Kevin Swersky, Kshiteej Mahajan, Laura Culp, Lechao Xiao, Maxwell L. Bileschi, Noah Constant, Roman Novak, Rosanne Liu, Tris Warkentin, Yundi Qian, Yamini Bansal, Ethan Dyer, Behnam Neyshabur, Jascha Sohl-Dickstein, and Noah Fiedel. Beyond human data: Scaling self-training for problem-solving with language models. *Transactions on Machine Learning Research*, 2024. URL https://openreview.net/forum?id=lNAyUngGFK.

Rahul Singhal, Karime Maamari, and Pradyumna Tambwekar. RLPE: Teaching LLMs to prompt with reinforcement learning, October 2026. URL https://arxiv.org/abs/2610.19062. Version 1, October 13, 2026.

Jake Snell, Kevin Swersky, and Richard S. Zemel. Prototypical networks for few-shot learning. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL https://arxiv.org/abs/1703.05175v2.

Qi Sun, Edoardo Cetin, and Yujin Tang. Transformer-Squared: Self-adaptive LLMs. In *International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=dh4t9qmcvK.

Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei Efros, and Moritz Hardt. Test-time training with self-supervision for generalization under distribution shifts. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pp. 9229–9248. PMLR, 2020.

Sebastian Thrun and Lorien Pratt (eds.). *Learning to Learn*. Kluwer Academic Publishers, 1998. doi: 10.1007/978-1-4615-5529-2.

Ruoyao Wang, Peter Jansen, Marc-Alexandre Côté, and Prithviraj Ammanabrolu. ScienceWorld: Is your agent smarter than a 5th grader? In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, 2022.

Yike Wang, Huaisheng Zhu, Zhengyu Hu, Yige Yuan, Zhengyu Chen, Shakti Senthil, Hannaneh Hajishirzi, Yulia Tsvetkov, Noah A. Smith, Pradeep Dasigi, and Teng Xiao. Rethinking the evaluation of harness evolution for agents, October 2026a. URL https://arxiv.org/abs/2607.12227v4. Version 4, 5 October 2026.

Zekai Wang, Yingqiang Ge, Zekun Wang, Hai Wang, Yuhui Xu, Joshua Frandsen, Shancong Fu, Ashia C. Wilson, and Chandan K. Reddy. VERSE: Verified self-evolving optimizer for agent harnesses, 2026b. URL https://arxiv.org/abs/2610.02616v1. Version 1, 2 October 2026.

Zekun Wang, Ethan Haarer, Tianyi Zhu, Zhiyi Dai, and Christopher J. MacLellan. Deep taxonomic networks for unsupervised hierarchical prototype discovery. In *Advances in Neural Information Processing Systems*, 2025a. URL https://arxiv.org/abs/2509.23602.

Zekun Wang, Anant Gupta, Zihan Dong, and Christopher J. MacLellan. Self-consolidating language models: Continual knowledge incorporation from context, May 2026c. URL https://arxiv.org/abs/2605.07076v2. Version 2, 12 May 2026.

Zhaoyang Wang, Weilei He, Zhiyuan Liang, Xuchao Zhang, Chetan Bansal, Ying Wei, Weitong Zhang, and Huaxiu Yao. CREAM: Consistency regularized self-rewarding language models. In *International Conference on Learning Representations*, 2025b. URL https://openreview.net/forum?id=Vf6RDObyEF.

Olga Wichrowska, Niru Maheswaranathan, Matthew W. Hoffman, Sergio Gómez Colmenarejo, Misha Denil, Nando de Freitas, and Jascha Sohl-Dickstein. Learned optimizers that scale and generalize. In *Proceedings of the 34th International Conference on Machine Learning*, 2017. URL https://arxiv.org/abs/1703.04813.

Zhiheng Xi, Yiwen Ding, Wenxiang Chen, Boyang Hong, Honglin Guo, Junzhe Wang, Xin Guo, Dingwen Yang, Chenyang Liao, Wei He, Songyang Gao, Lu Chen, Rui Zheng, Yicheng Zou, Tao Gui, Qi Zhang, Xipeng Qiu, Xuanjing Huang, Zuxuan Wu, and Yu-Gang Jiang. AgentGym: Evaluating and training large language model-based agents across diverse environments. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 27914–27961, 2025. doi: 10.18653/v1/2025.acl-long.1355. URL https://aclanthology.org/2025.acl-long.1355/.

Emily Xiao, Yixiao Zeng, Ada Chen, Chin-Jou Li, Amanda Bertsch, and Graham Neubig. Prompt-MII: Meta-learning instruction induction for LLMs. In *International Conference on Learning Representations*, 2026. URL https://arxiv.org/abs/2510.16932.

Sang Michael Xie, Aditi Raghunathan, Percy Liang, and Tengyu Ma. An explanation of in-context learning as implicit Bayesian inference. In *International Conference on Learning Representations*, 2022. URL https://arxiv.org/abs/2111.02080.

Chejian Xu, Shujian Zhang, Wenxuan Zhou, John Lambert, Beck Chen, Bo Li, Andrew Hard, and Rajiv Mathews. Evo^2: Evolving the evolver through weight-harness co-evolution, October 2026. URL https://arxiv.org/abs/2610.13025. Version 1, October 9, 2026.

Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *International Conference on Learning Representations*, 2024. URL https://arxiv.org/abs/2309.03409.

Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. WebShop: Towards scalable real-world web interaction with grounded language agents. In *Advances in Neural Information Processing Systems*, 2022.

Weiran Yao, Shelby Heinecke, Juan Carlos Niebles, Zhiwei Liu, Yihao Feng, Le Xue, Rithesh Murthy, Zeyuan Chen, Jianguo Zhang, Devansh Arpit, Ran Xu, Phil Mui, Huan Wang, Caiming Xiong, and Silvio Savarese. Retroformer: Retrospective large language agents with policy gradient optimization. In *International Conference on Learning Representations*, 2024. URL https://arxiv.org/abs/2308.02151.

Junjie Ye, Zhengyin Du, Xuesong Yao, et al. ToolHop: A query-driven benchmark for evaluating large language models in multi-hop tool use. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025.

Mingzhang Yin, George Tucker, Mingyuan Zhou, Sergey Levine, and Chelsea Finn. Meta-learning without memorization. In *International Conference on Learning Representations*, 2020. URL https://arxiv.org/abs/1912.03820.

Aohan Zeng, Mingdao Liu, Rui Lu, Bowen Wang, Xiao Liu, Yuxiao Dong, and Jie Tang. AgentTuning: Enabling generalized agent abilities for LLMs. In *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 3053–3077, 2024. doi: 10.18653/v1/2024.findings-acl.181. URL https://aclanthology.org/2024.findings-acl.181/.

Alvin Zhang, Xuecheng Liu, Zixuan Wang, Fahim Tajwar, Daman Arora, Ruslan Salakhutdinov, Daniel Khashabi, Yuda Song, and Andrea Zanette. Harness learning enables generalizable test-time adaptation, September 2026a. URL https://arxiv.org/abs/2609.35738. Version 1, September 28, 2026.

Guibin Zhang, Leo Lu, Fangzhou Xie, Kang Zhu, Junhao Wang, Zhifei Xie, Zhaochen Yu, Zihang Liu, Zhongxiang Sun, Qiankun Li, Yue Liao, Heng Chang, Xiaobin Hu, Qibing Ren, Wangchunshu Zhou, Chuanrui Hu, Yafeng Deng, and Shuicheng Yan. JIT-Agent: Scaling harness intelligence via just-in-time harness evolution, September 2026b. URL https://arxiv.org/abs/2608.25593. Version 2, 3 September 2026.

Hangfan Zhang, Shao Zhang, Kangcong Li, Chen Zhang, Yang Chen, Yiqun Zhang, Lei Bai, and Shuyue Hu. Self-Harness: Harnesses that improve themselves, 2026c. URL https://arxiv.org/abs/2606.09498. arXiv preprint, version 3, 20 August 2026.

Tunyu Zhang, Hao Wang, Kai Xu, and Dimitris N. Metaxas. Turbo Harness: Instance-adaptive harness optimization, September 2026d. URL https://arxiv.org/abs/2609.40330. Version 1, September 30, 2026.

Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. Large language models are human-level prompt engineers. In *International Conference on Learning Representations*, 2023. URL https://arxiv.org/abs/2211.01910.

Runchuan Zhu, Bowen Jiang, Lingrui Mei, Fangkai Yang, Lu Wang, Haoxiang Gao, Fengshuo Bai, Pu Zhao, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. AdaptFlow: Adaptive workflow optimization via meta-learning. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pp. 3287–3302. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.findings-emnlp.175. URL https://aclanthology.org/2025.findings-emnlp.175/.

Yuxin Zuo, Kaiyan Zhang, Li Sheng, Shang Qu, Ganqu Cui, Xuekai Zhu, Haozhan Li, Yuchen Zhang, Xinwei Long, Ermo Hua, Biqing Qi, Youbang Sun, Zhiyuan Ma, Lifan Yuan, Ning Ding, and Bowen Zhou. TTRL: Test-time reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 38, pp. 145667–145691, 2025. doi: 10.52202/085713-4376.

Table 3: The notation of Section 3 and Appendix A.

Symbol	Meaning
b, b'	A benchmark, and another training benchmark in the transfer step
$\rho(b)$	The population of benchmarks
H_0	The initial harness of a benchmark
$c \sim q_b(c)$, c'	A context of b, $c = \{I, H_0, \tau\}$ with the instruction I and a support set τ of failure trajectories under H_0, and a context of b'
θ, θ_t	The weights of the proposer, and their value in round t of meta-training
$e \sim p_\theta(e \mid c)$, $H = H_0 \oplus e$	An edit the proposer writes for c, and the edited harness
$r_b(H)$, $r_b(e)$	The success rate of the executor on the query set of b under H, and under $H_0 \oplus e$, the reward of e
$p_\theta(e' \mid c';\, c, e)$	The distribution of edits for c' after the proposer learns from (c, e)
$p_\theta(e' \mid c, e, c')$	The distribution of edits for c' in the transfer prompt after (c, e), Equation (5)
$T_\theta(c, e)$	The transfer of (c, e), Equation (2)
$J(\theta)$	The joint objective, Equation (3)
z	The latent fix strategy of the hierarchical Bayesian model
R, N	The rounds of meta-training, and the contexts per round
K, K'	The candidate edits per context, and the transfer edits per candidate
e_j, e'_{jk}, c'_k, b'_k	A candidate edit, its k-th transfer edit, and the context and benchmark of that transfer edit
$\hat{r}$, $\hat{T}$	A measured success rate, and the measured transfer
$\mathcal{L}(\theta)$	The training loss, Equation (6)
$s(c, e_j)$	The ranking score, Equation (9)
$\tilde{r}_b$, $\tilde{T}$, $\tilde{r}_{b'_k}$	A measured success rate and transfer, standardized over the candidates of c, and $\hat{r}_{b'_k}$ over the transfer edits on b'_k
$s'(e'_{jk})$	The transfer reward of a transfer edit, Algorithm 1
M, M', $\mathrm{top}_M(s)$	The candidates per context kept as adaptation examples and those of them that add transfer examples, and the indices of the M candidates with the highest $s(c, e_j)$
$\mathcal{D}$	The adaptation and transfer examples of a round, on which SFT trains the proposer

Adam Zweiger, Jyothish Pari, Han Guo, Yoon Kim, and Pulkit Agrawal. Self-adapting language models. In *Advances in Neural Information Processing Systems*, 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/6b41e04c41726e2a60e456d0a2b961ab-Abstract-Conference.html.

A Derivation of the Training Loss in Equation (6)

We write the objective as an expectation over both edits, substitute the in-context estimate of Equation (5) and derive the gradient and its Monte Carlo estimate. We then give the ranking score used in practice. Table 3 lists the notation.

The objective over both edits. Writing out the transfer of Equation (2) in Equation (3) gives

$$J(\theta) = \mathbb{E}_{b,c}\, \mathbb{E}_{b',c'} \sum_{e,e'} p_\theta(e \mid c)\, p_\theta(e' \mid c';\, c, e)\, r_b(e)\, r_{b'}(e'). \tag{7}$$

Substituting Equation (5) gives an objective in which both edits come from the proposer at the same weights:

$$J(\theta) \approx \mathbb{E}_{b,c}\, \mathbb{E}_{b',c'} \sum_{e,e'} p_\theta(e \mid c)\, p_\theta(e' \mid c, e, c')\, r_b(e)\, r_{b'}(e'). \tag{8}$$

Gradient. In Equation (8), only the probabilities $p_\theta(e \mid c)$ and $p_\theta(e' \mid c, e, c')$ depend on θ, and the gradient of their product is the product times the sum of their log-gradients, so the gradient

has one term for each edit:

$$\nabla_\theta J \approx \mathbb{E}_{b,c}\,\mathbb{E}_{b',c'} \sum_{e,e'} p_\theta(e \mid c)\, p_\theta(e' \mid c, e, c')\, r_b(e)\, r_{b'}(e') \Big[\underbrace{\nabla_\theta \log p_\theta(e \mid c)}_{\text{(A) first edit}} + \underbrace{\nabla_\theta \log p_\theta(e' \mid c, e, c')}_{\text{(B) second edit}} \Big].$$

Term (A). The factor $\nabla_\theta \log p_\theta(e \mid c)$ does not depend on e', so summing over e' first gives $\sum_{e'} p_\theta(e' \mid c, e, c')\, r_{b'}(e') = \mathbb{E}_{e' \sim p_\theta(\cdot \mid c, e, c')}\big[r_{b'}(e')\big]$, and averaging over b' and c' gives the transfer $T_\theta(c, e)$ of Equation (2), computed through the transfer prompt. Term (A) is therefore

$$\text{(A)} = \mathbb{E}_{b,c}\,\mathbb{E}_{e \sim p_\theta(\cdot \mid c)} \Big[r_b(e)\, T_\theta(c, e)\, \nabla_\theta \log p_\theta(e \mid c) \Big],$$

the gradient of the adaptation examples.

Term (B). Writing the sums as expectations gives

$$\text{(B)} = \mathbb{E}_{b,c}\,\mathbb{E}_{e \sim p_\theta(\cdot \mid c)}\,\mathbb{E}_{b',c'}\,\mathbb{E}_{e' \sim p_\theta(\cdot \mid c, e, c')} \Big[r_b(e)\, r_{b'}(e')\, \nabla_\theta \log p_\theta(e' \mid c, e, c') \Big],$$

the gradient of the transfer examples.

Monte Carlo estimate. At $\theta = \theta_t$, we replace each expectation over edits by samples from the proposer at θ_t: $e_j \sim p_{\theta_t}(\cdot \mid c)$ for $j = 1, \ldots, K$, and $e'_{jk} \sim p_{\theta_t}(\cdot \mid c, e_j, c'_k)$ for contexts c'_k of benchmarks $b'_k \neq b$, $k = 1, \ldots, K'$. We replace each success rate by its measured value $\hat{r}$, the transfer by $\hat{T}(c, e_j) = \frac{1}{K'} \sum_k \hat{r}_{b'_k}(e'_{jk})$ and the average over contexts by the average over the N contexts of the round. Terms (A) and (B) become

$$\frac{1}{N} \sum_c \frac{1}{K} \sum_{j=1}^{K} \hat{r}_b(e_j) \Big[\hat{T}(c, e_j)\, \nabla_\theta \log p_\theta(e_j \mid c) + \frac{1}{K'} \sum_{k=1}^{K'} \hat{r}_{b'_k}(e'_{jk})\, \nabla_\theta \log p_\theta(e'_{jk} \mid c, e_j, c'_k) \Big] \Big|_{\theta = \theta_t},$$

which is $-\nabla_\theta \mathcal{L}(\theta_t)$ for the loss $\mathcal{L}$ of Equation (6). The weights $\hat{r}$ and $\hat{T}$ enter $\mathcal{L}$ as constants, because the dependence of the transfer on θ through the second edit is already carried by term (B).

Ranking score. In practice, the K candidates e_j of a context c are ranked by

$$s(c, e_j) = \tilde{r}_b(e_j) + \tilde{T}(c, e_j), \tag{9}$$

where $\tilde{r}_b(e_j)$ and $\tilde{T}(c, e_j)$ are $\hat{r}_b(e_j)$ and $\hat{T}(c, e_j)$ standardized over the K candidates of c, which makes each a z-score. The reward and the transfer are measured on different benchmarks, whose initial harnesses solve different shares of their tasks and whose success rates spread differently across candidates, so we standardize each over the candidates of the context. Ranking by the weight $\hat{r}_b(e_j)\, \hat{T}(c, e_j)$ of Equation (6) is ranking by $\log \hat{r}_b(e_j) + \log \hat{T}(c, e_j)$, and near the means over the candidates each logarithm is approximately affine in its success rate, as a z-score is. The product therefore ranks the candidates approximately as a weighted sum of the two z-scores, and Equation (9) gives the two the same weight. We measure both as gains over the initial harnesses, which gives the same z-scores, since within a context $\hat{r}_b(H_0)$ and the transfer contexts are the same for every candidate.

B Experimental Details

Benchmarks and contexts. Each benchmark has a fixed query set of 50 tasks, and its support pool holds the executor's failures under H_0 on other tasks. The tasks of a training benchmark are split in half into a meta-train split and a meta-test split before the query sets and support pools are drawn, and the unseen benchmarks are used only for meta-evaluation (Table 4). BabyAI runs as text through the environment server of AgentGym (Xi et al., 2025). A context holds the instruction to the proposer, H_0 and five failure trajectories drawn at random with a fixed seed from a support pool, each compiled to one line per step.

Table 4: Number of tasks in the meta-train and meta-test splits of each benchmark.

Benchmark	Meta-train	Meta-test
WebShop	5,500	5,500
DBBench	2,401	2,402
ALFWorld	1,583	1,567
Gridworld	–	1,000
BIRD	–	4,714
BabyAI	–	900
ScienceWorld	–	4,639
TextWorld	–	1,000
ToolHop	–	498

Executor. The executor is Qwen3.5-9B with thinking disabled and greedy decoding, with at most 50 steps per episode and 2,048 tokens per turn, except on TextWorld, where an episode runs up to 100 steps with replies of up to 100 tokens.

Proposer. The proposer is Qwen3.5-9B with thinking enabled, sampled at temperature 1.0 and top-p 0.95 with up to 32,768 tokens, in meta-training and in evaluation. Its reasoning follows the four steps that the prompt asks for, from what went wrong in each trajectory to the property of the harness that allowed it (Appendix D). For SFT, we fine-tune it on edits that DeepSeek-V4.1-Flash writes at temperature 1.0 for contexts drawn from the meta-train splits of the three training benchmarks. A reply is kept if its edit is legal, its reasoning has the four steps and the edited harness solves at least one of the five failed tasks of its context when they are run again, a stricter form of the rerun filter of Harness-R1 (Shao et al., 2026). The kept replies are balanced to 333 per benchmark, 999 in total, and we fully fine-tune the proposer on them for two epochs at learning rate 10^{-5} with batch size 16.

Meta-training. Meta-training runs three rounds with five contexts per training benchmark per round. The proposer samples 30, 10 and 10 candidate edits per context in the three rounds. Each candidate is scored on the 50 query tasks of its benchmark, and the transfer edits written after it on three contexts of each of the two other training benchmarks, held out from the contexts used for adaptation. The reward and the transfer are gains in solved query tasks over H_0, each standardized over the candidates of a context, and the ranking score is their sum. The top 10, 3 and 3 candidates per context become adaptation examples, and the top 5, 2 and 2 of these each add one transfer example per other training benchmark. Fine-tuning uses AdamW with a constant learning rate of 10^{-4} and no warmup, two epochs per round, 8 examples per optimizer step, gradient norm clipping at 2.0 and sequences of up to 64,000 tokens.

LoRA. Each round trains a new LoRA adapter (Hu et al., 2022) on top of the frozen adapters of the earlier rounds, with rank 64, $\alpha = 128$ and dropout 0.05 on the query, key, value and output projections of the attention layers and the gate, up and down projections of the multilayer perceptron layers. GRPO and Dr. GRPO train the same adapters with dropout disabled.

Hyperparameter search. We swept the hyperparameters of meta-training over the values in Table 5 and use the values in bold, chosen by the success rate on the query sets of the meta-train splits.

Evaluation. We evaluate each benchmark with nine edits, three sampled for each of three contexts drawn at random with a fixed seed, and each edited harness runs once on the 50 query tasks. H_0 + failures gives one harness per context, and each of these runs three times. We report the mean of the benchmark's metric over the nine edits, or the nine runs of H_0 + failures, and its standard error, the standard deviation over the nine divided by 3. ScienceWorld scores an episode -100 when the executor focuses on a wrong object, and we count such an episode as 0.

Table 5: Hyperparameters swept for meta-training, with the values used in bold. Batch size counts the examples per optimizer step.

Setting	Parameter	Search space
LeAH and ordinary RL (Qwen3.5-9B)	Learning rate	[5e-5, **1e-4**, 2e-4, 3e-4]
	Batch size	[4, **8**, 16]
	Epochs per round	[1, **2**, 3]
	LoRA dropout	[0, **0.05**, 0.1]
LeAH and ordinary RL (Ministral 3 8B)	Learning rate	[**5e-5**, 1e-4]
GRPO and Dr. GRPO	Learning rate	[**1e-5**, 2e-5, 5e-5, 1e-4]
	Batch size	[8, **16**, 32]

Table 6: Compute and training examples of meta-training over the three rounds for the proposers of Tables 2 and 9. Executor episodes, proposer generations and training examples are given as those of the adaptation step plus those of the transfer step. An episode is one run of the executor on one query task, a generation is one reply of the proposer and the training examples are the top-ranked examples that the proposer is fine-tuned on.

Method	Executor episodes	Proposer generations	Training examples	GPU-hours
Ordinary RL	37,500 + 0	750 + 0	240 + 0	26.1
GPU-hour-matched ordinary RL	180,000 + 0	3,600 + 0	240 + 0	93.0
Training-data-matched LeAH	37,500 + 225,000	750 + 4,500	105 + 120	95.5
Joint (LeAH)	37,500 + 225,000	750 + 4,500	240 + 270	97.7

C Compute Resources

All training and model inference run on NVIDIA H200 GPUs, including the replies of the executor in every episode. Table 6 gives the executor episodes, proposer generations, training examples and GPU-hours of meta-training for ordinary reinforcement learning, GPU-hour-matched ordinary reinforcement learning, training-data-matched LeAH and LeAH. Ordinary reinforcement learning, training-data-matched LeAH and LeAH sample the same number of candidate edits per context, and the additional episodes and generations of LeAH and training-data-matched LeAH come from the six transfer edits that they write for each candidate edit. GPU-hour-matched ordinary reinforcement learning instead samples more candidate edits per context (Appendix G.2). Each method is fine-tuned only on its top-ranked examples, and LeAH trains on about twice as many as ordinary reinforcement learning, about half of them transfer examples. The benchmark environments run on a CPU machine with two Intel Xeon Platinum 8559C processors (192 vCPUs) and 1.5 TB of memory, and episodes run in parallel across tasks, contexts and edits. These are the costs of meta-training alone. At meta-evaluation and deployment, every proposer writes each edit from one context in a single generation, with no candidate search, transfer step or weight update.

D Prompts and Trajectory Compilation

This appendix gives the prompt from which the proposer writes an edit, the transfer prompt of meta-training and the rule that compiles failure trajectories into the text of a context.

Proposer prompt. The system prompt states the task and the three levels of reasoning, and the user turn holds the context (Listing 1). The user turn asks for four reasoning steps, states the constraints, gives the current harness, describes the hooks that an edit may add, gives the five compiled failure trajectories, and ends with the output format. The text is the same for every benchmark except for the harness fields, the trajectories, and one sentence of the hooks reference that states how the executor's actions reach the hooks, written from the benchmark's interface.

```
[SYSTEM]
```

```
You improve the harness of a frozen language-model agent: the text it is given and the code that runs around it
      ↪ in its loop. You are shown failed trajectories of that agent on tasks of one environment and you edit
      ↪ the harness so the same agent succeeds there. You reason in a fixed compact form, in three levels: the
      ↪ task level, what went wrong in each trajectory; the environment level, the failure mode this agent
      ↪ shows in this environment, which is what the trajectories are evidence of; the harness level, the
      ↪ property of the harness that allowed it, stated as a principle that would hold in any environment. The
      ↪ edit should generalize across the three levels: it addresses what went wrong in the trajectories shown,
      ↪  it holds for the other tasks of this environment, and it carries the harness-level principle. It is
      ↪ written once, in the output, after the reasoning.

[USER]
You are given (1) the current harness of an agent: the text fields its frozen executor model reads, and an empty
      ↪  hooks file; (2) the hooks API, the code layer you may add to the agent's loop; (3) 5 failed
      ↪ trajectories of that executor under this harness, on 5 different tasks of the same environment. The
      ↪ harness is the only thing you can change.

Your job: work out why these 5 trajectories failed, find what the failures have in common, and edit the harness
      ↪ so that this executor would have succeeded on them and on the other tasks of this environment that fail
      ↪  the same way.

How the edit is judged: the same executor is re-run under your harness on these 5 tasks and on other tasks of
      ↪ this environment. More successes is better.

Procedure. Do these steps in your reasoning, in order, each headed by its name, before you write the edit.
Task level. For each trajectory, #0 to #4: the step where it went wrong and why, in one or two sentences, from
      ↪ the trajectory's own evidence.
Environment level. The failure mode or modes this executor shows in this environment: what recurs across the 5
      ↪ and would recur on other tasks here. State each as behaviour under this harness, not as a fact about
      ↪ one task, and say which trajectories show it.
Harness level. One level up: what property of the harness let that behaviour happen: what the executor was not
      ↪ told, not made to check, not able to see, not able to do, or not stopped from doing. State it as a
      ↪ principle that would hold in any environment where an agent behaves this way. Then say how that
      ↪ principle is instantiated here, in this environment's interface, and which part of the harness (text, a
      ↪  hook point, a tool, a model call) carries it.
Decision. One sentence per part of the harness you will change, saying what it does and which mode it addresses.
      ↪  Then close the reasoning; the edit itself is written once, in the output, after </think>.

Constraints:
- Keep the executor's interface: the action format, the tool-call conventions, and any placeholders a field
      ↪ already contains.
- Do not name the benchmark, dataset or environment, and do not encode task-instance specifics: task indices,
      ↪ exact answers, ground-truth queries, or object, product, file, table, document names or values copied
      ↪ from the traces. Describe behaviour, not instances.
- One edit per reply.

Current harness fields (the text the executor is given):
[THE HARNESS FIELDS OF THE BENCHMARK, AS JSON]

## Hooks: code that runs inside the executor's agent loop
You may deliver a Python file defining class Hooks(BaseHooks); every method is optional. The loop calls each
      ↪ method at a fixed point of every turn; a method can change what the executor sees, what it does, or
      ↪ what it can do:

```python
class Hooks(BaseHooks):
    def system_prompt(self, assembled):           # str -> str: the final system prompt, before the first turn
        return assembled
    def before_llm(self, msgs, state):            # list of {"role","content"} dicts -> the list the model
      ↪ reads this turn; keep msgs[0], the system message, in place
        return msgs
    def after_llm(self, message, state):          # the assistant message dict {"role", "content": str|None, "
      ↪ tool_calls": list|None} -> the message the loop acts on; a tool call written in "content" as text may
      ↪ be turned into a real entry of "tool_calls" ({"function": {"name", "arguments" (JSON str)}}); do not
      ↪ call str methods on the dict
        return message
    def before_tool(self, name, args, state):     # the tool call about to run -> args (a dict, possibly
      ↪ rewritten); or a str, which cancels the call and is shown to the executor as its result; or (str,
      ↪ synthetic_result) to cancel and answer with a result of your own
        return args
    def after_tool(self, name, args, obs, state): # the tool's output string -> the string the executor sees
        return obs
    def extra_tools(self):                        # -> [{name, description, input_schema}]: tools added to the
      ↪ executor's tool list
        return []
    def run_tool(self, name, args, env, state):   # -> str: your implementation of an added tool (env has no
      ↪ container in this environment)
        return "..."
    def on_turn_end(self, turn, state):           # -> str|None: text appended to the next user turn
        return None
```
In this environment [HOW THE EXECUTOR'S ACTIONS REACH THE HOOKS IN THIS BENCHMARK].
```

```
self.llm(prompt, max_tokens=1024) is available in every method: a call to the executor's own model with a prompt
      ↪  of your choice, returning its text. One call costs one turn of the episode's turn budget (turns + llm
      ↪ calls <= max_turns); model and temperature are the episode's.

state is a dict that belongs to you, empty at the start of every episode and kept across its turns. before_llm
      ↪ sees the full message list every turn, so a transform applied there each turn is a durable rewrite of
      ↪ the history.

Rules enforced by a static audit (a violation rejects the proposal): imports limited to re, json, math, string,
      ↪ textwrap, difflib, collections, itertools, functools, heapq, bisect, statistics, copy, fnmatch, shlex;
      ↪ no file, network or process access. Import at the top of the file what you use; a NameError at call
      ↪ time makes that call a no-op. A method that raises is a no-op for that call and is logged.

## Failure traces (5 failed trajectories under the current harness, one line per step)

[THE FIVE COMPILED FAILURE TRAJECTORIES]

## Output format
Continue your reasoning inside the already-opened thinking block, close it with </think>, then output exactly
      ↪ one <edit> block containing one JSON object:
<edit>
{"fields": {FIELD_NAME: FULL_NEW_TEXT}, "rationale": TEXT}
</edit>
Keys of fields must be among [THE NAMES OF THE EDITABLE FIELDS, AS A JSON LIST]; a field you leave out is
      ↪ unchanged; each value is the complete new text of that field and must not be empty; fields may be {} if
      ↪  you change only the hooks. If you use code, the hooks file follows the </edit> block as one fenced ```
      ↪ python block, the full source of the file (a class Hooks(BaseHooks) obeying the audit rules above), and
      ↪  nothing else follows it. An edit that leaves every field as it was is rejected. No markdown inside the
      ↪  <edit> block.
```

Listing 1: The proposer prompt. Bracketed capitals mark the parts that differ between contexts.

Transfer prompt. The transfer prompt (c, e, c') of Section 3.3 places one finished exchange before a new context (Listing 2). The exchange is the system prompt, the context c of the source benchmark as the user turn and the proposer's whole reply to it, with its reasoning and its edit e, as the assistant turn. The context c' of the target benchmark follows as the next user turn with no added text, and the reply to it is the transfer edit e'.

```
[SYSTEM]     the system prompt of Listing 1
[USER]       context c of benchmark b
[ASSISTANT]  the proposer's reply to c: its reasoning, then the edit e
[USER]       context c' of another training benchmark b'
[ASSISTANT]  (generated: the reasoning and the transfer edit e')
```

Listing 2: The transfer prompt. Each context is a user turn of Listing 1.

Trajectory compilation. Each failure trajectory of a context is compiled to text by a fixed rule, much as Harness-R1 compacts failed episodes into a failure packet (Shao et al., 2026). A compiled trajectory hides the task identifier and opens with the outcome, the reward, the number of steps and the environment's first message (Listing 3). It then has one line per step, with what the executor wrote, the action or tool call that the loop took from it and what the environment returned, and it ends with the reason the episode ended. Repeated steps and observations are written once, and long parts are clipped so that every context fits the training length.

```
## trajectory #[INDEX] | FAIL | reward [REWARD] | steps [STEPS]
- task: [THE ENVIRONMENT'S FIRST MESSAGE] || [THE ADMISSIBLE ACTIONS, IF THE BENCHMARK LISTS THEM]
[1] > prose: [WHAT THE EXECUTOR WROTE] | call: [THE TOOL CALL THE LOOP TOOK] | env: [WHAT THE ENVIRONMENT
      ↪ RETURNED]
[2-4] (x3) > [THREE IDENTICAL CONSECUTIVE STEPS, WRITTEN ONCE]
[5] > call: [THE TOOL CALL] | env: (same as step 1)
...
- ended: [WHY THE EPISODE ENDED, SUCH AS THE ENVIRONMENT'S VERDICT OR THE STEP LIMIT]
```

Listing 3: The format of one compiled failure trajectory. Bracketed capitals mark the recorded text.

Table 7: Other proposers and executors. Upper groups: Ministral 3 8B and Qwen3.8-27B as the proposer, with the same teacher edits and recipes, and with the executor and H_0 of Table 1. Lower groups: the final proposers of Table 2, without further training, for Ministral 3 8B and gpt-oss-20b as the executor, each with its own H_0. Cells as in Table 1. Bold and underline mark the best and second-best proposer within each upper group, and bold marks the better proposer within each lower group.

	Training benchmarks (held-out tasks)				Unseen benchmarks						
Proposer	WebShop	DBBench	ALFWorld	Avg.	Gridworld	BIRD	BabyAI†	SciWorld†	TextWorld†	ToolHop	Avg.
H_0	12.0	42.0	52.0	35.3	74.0	54.0	73.6	22.6	38.2	46.0	51.4
Ministral 3 8B proposer, Qwen3.5-9B executor (as in Section 4)											
SFT	24.2$_{\pm 1.1}$	47.8$_{\pm 6.1}$	52.0$_{\pm 7.4}$	41.3$_{\pm 3.2}$	60.4$_{\pm 14.5}$	38.0$_{\pm 7.2}$	**79.2**$_{\pm 2.3}$	26.7$_{\pm 1.2}$	35.0$_{\pm 2.3}$	**48.4**$_{\pm 1.9}$	48.0$_{\pm 2.8}$
Ordinary RL	24.2$_{\pm 1.2}$	52.4$_{\pm 0.8}$	56.4$_{\pm 3.3}$	44.4$_{\pm 1.2}$	79.3$_{\pm 8.4}$	**44.9**$_{\pm 6.1}$	78.9$_{\pm 1.6}$	**30.6**$_{\pm 1.2}$	39.7$_{\pm 2.2}$	44.0$_{\pm 5.6}$	52.9$_{\pm 2.0}$
Joint (LeAH)	**28.2**$_{\pm 1.8}$	**55.8**$_{\pm 1.6}$	**58.9**$_{\pm 3.5}$	**47.6**$_{\pm 1.4}$	**90.0**$_{\pm 2.7}$	38.2$_{\pm 7.3}$	76.3$_{\pm 1.8}$	29.4$_{\pm 2.2}$	**45.8**$_{\pm 2.1}$	48.0$_{\pm 1.0}$	**54.6**$_{\pm 1.4}$
Qwen3.8-27B proposer, Qwen3.5-9B executor (as in Section 4)											
SFT	27.8$_{\pm 1.4}$	52.9$_{\pm 1.5}$	59.8$_{\pm 3.0}$	46.8$_{\pm 1.2}$	71.1$_{\pm 11.3}$	50.7$_{\pm 0.9}$	75.6$_{\pm 5.0}$	31.3$_{\pm 3.4}$	41.7$_{\pm 2.1}$	46.0$_{\pm 5.8}$	52.7$_{\pm 2.4}$
Ordinary RL	**28.4**$_{\pm 1.1}$	54.7$_{\pm 1.0}$	**61.8**$_{\pm 3.0}$	**48.3**$_{\pm 1.1}$	86.0$_{\pm 4.1}$	47.3$_{\pm 1.2}$	75.7$_{\pm 1.8}$	37.0$_{\pm 3.0}$	**44.8**$_{\pm 2.1}$	53.3$_{\pm 0.7}$	57.4$_{\pm 1.0}$
Joint (LeAH)	25.6$_{\pm 1.4}$	**54.9**$_{\pm 0.9}$	61.1$_{\pm 3.1}$	47.2$_{\pm 1.2}$	**91.1**$_{\pm 1.5}$	**52.7**$_{\pm 1.5}$	**81.7**$_{\pm 2.1}$	**38.4**$_{\pm 2.9}$	39.1$_{\pm 2.2}$	**53.6**$_{\pm 1.1}$	**59.4**$_{\pm 0.8}$
Qwen3.5-9B proposer (as in Section 4), Ministral 3 8B executor											
H_0	8.0	32.0	22.0	20.7	88.0	38.0	9.8	6.3	8.4	42.0	32.1
Ordinary RL	**17.1**$_{\pm 1.5}$	39.3$_{\pm 5.2}$	28.9$_{\pm 1.8}$	28.4$_{\pm 1.9}$	70.2$_{\pm 3.3}$	**46.7**$_{\pm 0.9}$	69.3$_{\pm 2.5}$	**8.6**$_{\pm 2.0}$	**12.6**$_{\pm 3.6}$	35.1$_{\pm 1.5}$	40.4$_{\pm 1.0}$
Joint (LeAH)	16.2$_{\pm 2.0}$	**42.2**$_{\pm 2.3}$	**32.7**$_{\pm 2.8}$	**30.4**$_{\pm 1.4}$	**75.8**$_{\pm 2.0}$	42.9$_{\pm 1.6}$	**71.0**$_{\pm 1.2}$	**8.6**$_{\pm 1.5}$	11.2$_{\pm 1.7}$	**37.1**$_{\pm 4.7}$	**41.1**$_{\pm 1.0}$
Qwen3.5-9B proposer (as in Section 4), gpt-oss-20b executor											
H_0	12.0	42.0	26.0	26.7	56.0	22.0	78.5	5.9	46.2	30.0	39.7
Ordinary RL	25.8$_{\pm 1.2}$	**50.9**$_{\pm 1.4}$	39.1$_{\pm 2.2}$	**38.6**$_{\pm 0.9}$	40.4$_{\pm 8.5}$	22.0$_{\pm 5.0}$	76.1$_{\pm 1.2}$	7.3$_{\pm 1.9}$	**52.8**$_{\pm 2.3}$	31.3$_{\pm 4.9}$	38.3$_{\pm 1.9}$
Joint (LeAH)	**28.7**$_{\pm 1.9}$	45.6$_{\pm 5.8}$	**41.3**$_{\pm 3.0}$	38.5$_{\pm 2.3}$	**60.4**$_{\pm 7.4}$	**22.2**$_{\pm 2.1}$	**78.1**$_{\pm 1.6}$	**8.2**$_{\pm 1.6}$	44.8$_{\pm 2.5}$	**35.6**$_{\pm 2.4}$	**41.6**$_{\pm 1.5}$

E Other Proposer and Executor Models

We test whether the results depend on the proposer model and whether a trained proposer transfers to other executors (Table 7). In the upper groups, Ministral 3 8B (Liu et al., 2026) and the larger Qwen3.8-27B (Qwen Team, 2026b) replace Qwen3.5-9B as the proposer, with the executor, H_0, contexts and evaluation of Table 1. Each SFT proposer is fine-tuned on the same 999 teacher edits (Appendix B), and meta-training runs both recipes with the hyperparameters of Qwen3.5-9B, except for a learning rate of 5×10^{-5} for Ministral 3 8B (Table 5). With both proposers, LeAH is above its SFT proposer and ordinary reinforcement learning on the unseen benchmarks (54.6 against 48.0 and 52.9 for Ministral 3 8B, 59.4 against 52.7 and 57.4 for Qwen3.8-27B), and on the training benchmarks it is above ordinary reinforcement learning for Ministral 3 8B (47.6% against 44.4%) and comparable to it for Qwen3.8-27B (47.2% against 48.3%). In the lower groups, the final proposers of LeAH and ordinary reinforcement learning in Table 2, meta-trained with Qwen3.5-9B as the executor, write edits without further training for two other executors, Ministral 3 8B and the larger gpt-oss-20b (OpenAI, 2025). They read contexts of each executor's own failures under H_0, and gpt-oss-20b runs at low reasoning effort to match the non-thinking mode of the other executors. On the training benchmarks, LeAH and ordinary reinforcement learning both improve on H_0, from 20.7% to 30.4% and 28.4% with Ministral 3 8B and from 26.7% to 38.5% and 38.6% with gpt-oss-20b. On the unseen benchmarks, both improve on H_0 with Ministral 3 8B (41.1 and 40.4 against 32.1), and with gpt-oss-20b, LeAH stays above H_0 (41.6 against 39.7) while ordinary reinforcement learning does not (38.3). This suggests that the adaptation rule that LeAH learns with one executor carries over to other executors, including a larger one.

Table 8: The joint objective optimized with GRPO and Dr. GRPO instead of the supervised loss on the top-ranked examples. Cells as in Table 1. Bold and underline mark the best and second-best proposer.

	Training benchmarks (held-out tasks)				Unseen benchmarks						
Proposer	WebShop	DBBench	ALFWorld	Avg.	Gridworld	BIRD	BabyAI†	SciWorld†	TextWorld†	ToolHop	Avg.
H_0	12.0	42.0	52.0	35.3	74.0	54.0	73.6	22.6	38.2	46.0	51.4
SFT	$21.1_{\pm 3.8}$	$52.9_{\pm 1.5}$	$55.6_{\pm 3.5}$	$43.2_{\pm 1.8}$	$\underline{80.4}_{\pm 9.6}$	$39.1_{\pm 6.9}$	$74.8_{\pm 2.5}$	$26.2_{\pm 2.8}$	$29.8_{\pm 6.6}$	$46.0_{\pm 5.0}$	$49.4_{\pm 2.5}$
GRPO	$\underline{22.4}_{\pm 3.3}$	$48.9_{\pm 6.3}$	$\mathbf{62.0}_{\pm 3.1}$	$44.4_{\pm 2.6}$	$80.0_{\pm 10.2}$	$41.1_{\pm 7.8}$	$73.2_{\pm 3.9}$	$24.9_{\pm 3.4}$	$34.2_{\pm 2.4}$	$\mathbf{50.2}_{\pm 1.1}$	$50.6_{\pm 2.3}$
Dr. GRPO	$20.9_{\pm 2.1}$	$\underline{54.7}_{\pm 1.4}$	$59.3_{\pm 3.7}$	$\underline{45.0}_{\pm 1.5}$	$79.8_{\pm 7.9}$	$\underline{45.6}_{\pm 5.7}$	$\underline{76.9}_{\pm 1.8}$	$\underline{28.6}_{\pm 3.0}$	$\underline{35.1}_{\pm 7.3}$	$\underline{49.8}_{\pm 1.2}$	$\underline{52.6}_{\pm 2.1}$
LeAH	$\mathbf{29.8}_{\pm 1.3}$	$\mathbf{55.3}_{\pm 2.2}$	$\underline{60.4}_{\pm 3.8}$	$\mathbf{48.5}_{\pm 1.5}$	$\mathbf{90.9}_{\pm 2.2}$	$\mathbf{54.7}_{\pm 0.7}$	$\mathbf{78.5}_{\pm 1.8}$	$\mathbf{32.8}_{\pm 3.0}$	$\mathbf{42.0}_{\pm 2.1}$	$48.4_{\pm 1.3}$	$\mathbf{57.9}_{\pm 0.8}$

F Optimizing the Joint Objective with GRPO

LeAH optimizes the joint objective by training with a supervised loss on the top-ranked examples of each context (Section 3.4). We also optimize it with GRPO (Shao et al., 2024) and Dr. GRPO (Liu et al., 2025), from the same SFT proposer and with the same rewards, rounds, contexts and candidates. The candidates of a context form one group, rewarded by the sum of the standardized reward and the standardized transfer, as in the ranking of LeAH. The transfer edits written after one top candidate for the three contexts of another training benchmark form a second kind of group, rewarded by their gain over H_0 standardized over the round, and these groups come from the same top 5, 2 and 2 candidates that give LeAH its transfer examples. The advantage of each completion is its reward minus the mean of its group, divided by the group's standard deviation in GRPO, while Dr. GRPO keeps the difference and normalizes the loss by a constant instead of the completion length. Both optimize one clipped objective with $\epsilon = 0.2$ and no Kullback-Leibler penalty, for one epoch per round, with the learning rate and batch size of Table 5.

We find both unstable to train in this setting, and both end below LeAH (Table 8). We attribute this to two causes. First, each round trains on a fixed set of samples. Each round samples and scores its candidates once, and the update then takes many minibatch steps on this fixed set, while GRPO is designed for small steps on fresh samples of the current policy. Most of its steps therefore use advantages computed for a proposer that has already moved, while the supervised loss of LeAH uses no advantages. Second, group-relative advantages amplify the noise of the rewards. The reward and the transfer of a candidate come from a single run of each query task, and standardizing them within the group of one context turns small differences between candidates into advantages of full size, whose sign a second run could reverse. Dr. GRPO, which does not divide by the standard deviation of the group, is slightly better but still below LeAH. Selecting the top-ranked candidates, as LeAH does following ReST (Gulcehre et al., 2023; Singh et al., 2024), is more robust to this noise. It keeps only the candidates that rank highest in their context and trains on them with a supervised loss, so noise in the rewards changes which edits are kept but not the direction of the update on any of them.

G Further Comparisons with Ordinary Reinforcement Learning

G.1 A Matched Number of Training Examples

We additionally compare LeAH with ordinary reinforcement learning at a matched number of training examples (Table 9). In Table 2, both objectives sample the same number of candidate edits per context and keep the same number of top-ranked adaptation examples, and LeAH also trains on transfer examples. Training-data-matched LeAH runs the adaptation and transfer steps of LeAH and keeps, for each context, only the top-ranked half of the adaptation examples and of the candidates that add transfer examples, rounded down, so that each round trains on no more examples than ordinary reinforcement learning (Table 6). Keeping the top-ranked examples keeps the selection of LeAH and changes only their number. Training-data-matched LeAH reaches 56.3 on the unseen benchmarks, against 53.5 for ordinary reinforcement learning and 57.9 for LeAH with all its examples, and is higher than ordinary reinforcement learning

Table 9: Ordinary reinforcement learning with about the GPU-hours of LeAH (GPU-hour-matched) and LeAH trained on no more examples per round than ordinary reinforcement learning (training-data-matched), with the two proposers of Table 2. Cells as in Table 1. Bold and underline mark the best and second-best proposer.

	Training benchmarks (held-out tasks)				Unseen benchmarks						
Proposer	WebShop	DBBench	ALFWorld	Avg.	Gridworld	BIRD	BabyAI†	SciWorld†	TextWorld†	ToolHop	Avg.
H_0	12.0	42.0	52.0	35.3	74.0	54.0	73.6	22.6	38.2	46.0	51.4
Ordinary RL	25.1$_{\pm 2.5}$	<u>55.1</u>$_{\pm 1.7}$	<u>62.7</u>$_{\pm 1.6}$	<u>47.6</u>$_{\pm 1.1}$	78.9$_{\pm 9.7}$	47.8$_{\pm 6.0}$	**78.6**$_{\pm 0.9}$	26.5$_{\pm 2.4}$	39.0$_{\pm 1.8}$	**50.4**$_{\pm 0.9}$	53.5$_{\pm 2.0}$
GPU-hour-matched ordinary RL	<u>26.4</u>$_{\pm 2.9}$	54.0$_{\pm 1.9}$	60.9$_{\pm 3.0}$	47.1$_{\pm 1.5}$	75.8$_{\pm 10.1}$	33.8$_{\pm 8.5}$	75.9$_{\pm 2.6}$	<u>30.3</u>$_{\pm 1.7}$	38.5$_{\pm 3.6}$	45.6$_{\pm 5.8}$	50.0$_{\pm 2.5}$
Training-data-matched LeAH	24.0$_{\pm 1.6}$	52.9$_{\pm 3.0}$	**65.1**$_{\pm 2.4}$	47.3$_{\pm 1.4}$	**92.0**$_{\pm 1.6}$	<u>52.9</u>$_{\pm 1.1}$	76.4$_{\pm 2.4}$	26.9$_{\pm 3.6}$	<u>39.6</u>$_{\pm 2.6}$	<u>49.8</u>$_{\pm 0.9}$	<u>56.3</u>$_{\pm 0.9}$
Joint (LeAH)	**29.8**$_{\pm 1.3}$	**55.3**$_{\pm 2.2}$	60.4$_{\pm 3.8}$	**48.5**$_{\pm 1.5}$	<u>90.9</u>$_{\pm 2.2}$	**54.7**$_{\pm 0.7}$	<u>78.5</u>$_{\pm 1.8}$	**32.8**$_{\pm 3.0}$	**42.0**$_{\pm 2.1}$	48.4$_{\pm 1.3}$	**57.9**$_{\pm 0.8}$

on four of the six benchmarks. On the training benchmarks, training-data-matched LeAH and ordinary reinforcement learning are comparable (47.3% and 47.6%).

G.2 A Matched Number of GPU-Hours

LeAH uses 97.7 GPU-hours of meta-training against 26.1 for ordinary reinforcement learning, mostly for the episodes and generations of its transfer edits (Table 6). To compare the two objectives at comparable compute, GPU-hour-matched ordinary reinforcement learning runs the first round as ordinary reinforcement learning does and then samples 105 instead of 10 candidate edits per context in each of the next two rounds, keeping the same number of top-ranked examples, which takes 93.0 GPU-hours. It reaches 50.0 on the unseen benchmarks, below ordinary reinforcement learning (53.5) and H_0 (51.4), and is lower than LeAH on all six benchmarks (Table 9). On the training benchmarks, it is comparable to ordinary reinforcement learning (47.1% and 47.6%). Training-data-matched LeAH uses comparable compute (95.5 GPU-hours) and trains on no more examples (225 against 240), and it is higher than GPU-hour-matched ordinary reinforcement learning on the unseen benchmarks (56.3 against 50.0) and comparable on the training ones (47.3% and 47.1%). Compute and the number of training examples therefore do not explain the gain of LeAH on the unseen benchmarks.

G.3 Fewer Failures in the Context

The proposers are meta-trained on contexts of five failures, and we also give the final proposers contexts with one or three of these failures (Table 10). On the training benchmarks, more failures help both proposers and LeAH more, from 37.0% with one failure to 48.5% with five, against 40.8% to 47.6% for ordinary reinforcement learning. On the unseen benchmarks, LeAH is higher than ordinary reinforcement learning on four, three and four of the six benchmarks with one, three and five failures. The averages are comparable with one and three failures (55.1 and 55.2, 51.3 and 51.0) mainly because of BIRD, where LeAH's edits vary much more with fewer failures, with standard errors of 5.7 and 8.7 against 0.7 with five. Without BIRD, LeAH is higher on average with every number of failures, and its lead grows with them (57.0 against 55.8 with one failure, 54.6 against 52.0 with three and 58.5 against 54.7 with five). Fewer failures show less of how a benchmark fails, and LeAH, whose edits more often use rules specific to the new benchmark (Section 5.4), gains more as the number of failures grows.

H Case Study

This appendix gives the protocol of the case study in Section 5.4 and expands its two findings: LeAH's edits more often state the step that completes a task as the new benchmark defines it (Appendix H.2), and its hooks read the executor's action where the new benchmark passes it (Appendix H.3). We then follow both over meta-training (Appendix H.4).

Table 10: Contexts with one, three or five failures for the final proposers of Table 2, which are meta-trained on contexts of five. Top: training benchmarks. Bottom: unseen benchmarks, with the average over the five benchmarks other than BIRD in the last column. Cells as in Table 1. Bold marks the better proposer at each number of failures.

		Training benchmarks (held-out tasks)			
Failures	Proposer	WebShop	DBBench	ALFWorld	Avg.
	H_0	12.0	42.0	52.0	35.3
1	Ordinary RL	**20.4**$_{\pm 3.4}$	**52.0**$_{\pm 1.1}$	**50.0**$_{\pm 1.7}$	**40.8**$_{\pm 1.3}$
	Joint (LeAH)	17.6$_{\pm 3.3}$	45.1$_{\pm 6.7}$	48.4$_{\pm 2.4}$	37.0$_{\pm 2.6}$
3	Ordinary RL	**24.9**$_{\pm 1.7}$	**55.1**$_{\pm 1.4}$	56.2$_{\pm 2.7}$	**45.4**$_{\pm 1.2}$
	Joint (LeAH)	**24.9**$_{\pm 3.2}$	53.1$_{\pm 1.4}$	**57.6**$_{\pm 2.2}$	45.2$_{\pm 1.4}$
5	Ordinary RL	25.1$_{\pm 2.5}$	55.1$_{\pm 1.7}$	**62.7**$_{\pm 1.6}$	47.6$_{\pm 1.1}$
	Joint (LeAH)	**29.8**$_{\pm 1.3}$	**55.3**$_{\pm 2.2}$	60.4$_{\pm 3.8}$	**48.5**$_{\pm 1.5}$

		Unseen benchmarks							
Failures	Proposer	Gridworld	BIRD	BabyAI^{+}	SciWorld^{+}	TextWorld^{+}	ToolHop	Avg.	Avg. w/o BIRD
	H_0	74.0	54.0	73.6	22.6	38.2	46.0	51.4	50.9
1	Ordinary RL	85.1$_{\pm 3.0}$	**52.2**$_{\pm 1.1}$	73.3$_{\pm 2.5}$	28.9$_{\pm 1.9}$	39.6$_{\pm 1.7}$	**52.2**$_{\pm 1.4}$	**55.2**$_{\pm 0.8}$	55.8$_{\pm 1.0}$
	Joint (LeAH)	**88.4**$_{\pm 2.5}$	45.1$_{\pm 5.7}$	**76.0**$_{\pm 1.7}$	**30.0**$_{\pm 1.5}$	**39.9**$_{\pm 1.6}$	50.9$_{\pm 1.1}$	55.1$_{\pm 1.1}$	**57.0**$_{\pm 0.8}$
3	Ordinary RL	70.9$_{\pm 11.2}$	**46.0**$_{\pm 5.9}$	**71.7**$_{\pm 3.6}$	28.8$_{\pm 2.0}$	**39.6**$_{\pm 2.0}$	48.9$_{\pm 0.8}$	51.0$_{\pm 2.2}$	52.0$_{\pm 2.4}$
	Joint (LeAH)	**81.1**$_{\pm 4.0}$	34.4$_{\pm 8.7}$	**71.7**$_{\pm 2.6}$	**30.7**$_{\pm 1.9}$	38.9$_{\pm 1.9}$	**50.7**$_{\pm 1.6}$	**51.3**$_{\pm 1.7}$	**54.6**$_{\pm 1.1}$
5	Ordinary RL	78.9$_{\pm 9.7}$	47.8$_{\pm 6.0}$	**78.6**$_{\pm 0.9}$	26.5$_{\pm 2.4}$	39.0$_{\pm 1.8}$	**50.4**$_{\pm 0.9}$	53.5$_{\pm 2.0}$	54.7$_{\pm 2.0}$
	Joint (LeAH)	**90.9**$_{\pm 2.2}$	**54.7**$_{\pm 0.7}$	78.5$_{\pm 1.8}$	**32.8**$_{\pm 3.0}$	**42.0**$_{\pm 2.1}$	48.4$_{\pm 1.3}$	**57.9**$_{\pm 0.8}$	**58.5**$_{\pm 1.0}$

H.1 Protocol

We read three sets of edits. The first is the final edits of the SFT proposer, ordinary reinforcement learning and LeAH on the six unseen benchmarks, nine per benchmark and proposer, together with the teacher's edits for the same contexts. The second is the edits of the two trained proposers for the unseen benchmarks after each round of meta-training. The third is the transfer edits of LeAH's training rounds, with the candidates they were written from and their success on the target benchmark. Each edit is read with the proposer's visible reasoning, the five failures it was written from and the episodes of the edited harness on the query tasks. For each edit, we record the rules it states, how it states the step that completes a task and where its hooks read the executor's action. On ScienceWorld, the first focus of an episode is right if the first `focus on` that the environment accepts raises the score.

H.2 The Step That Completes a Task

ScienceWorld names the object of each task and, in 43 of its 50 query tasks, asks the executor to focus on it first ("First, focus on the aluminum foil."). A focus on any other object ends the episode, and we count its score as 0 (Appendix B). Under H_0, the executor focuses on the right object first in 14 of the 50 episodes, and in 23 it never focuses.

LeAH's final edits state this step more often than those of ordinary reinforcement learning and the SFT proposer (Table 11). Five of its nine ScienceWorld edits say to focus on the task's object first, for example "Search the rooms and containers until you find it, then use 'focus on <task object>' " and the rule shown in Figure 3a. Two of the nine edits of ordinary reinforcement learning say so. Three of its other edits say to focus at the end, for example "When the task is complete, focus on the resulting object", and the edit of Figure 3a asks to first perform the step that completes the task, without saying that this step is the focus. The teacher states the rule in seven of its nine edits, and the SFT proposer, fine-tuned on the teacher's edits for the training benchmarks, in one of nine. LeAH thus learns in meta-training a rule that the teacher writes and that imitating the teacher did not teach the SFT proposer.

Table 11: The two findings over meta-training. Focus first: ScienceWorld edits, of 9, that say to focus on the task's object first or in the order the task gives. Right first focus: share of the 450 ScienceWorld episodes whose first focus is on the right object (14 of 50 under H_0). Tool-call field: edits whose hooks read the tool-call field, of 36 edits on ScienceWorld, BabyAI, TextWorld and Gridworld.

Proposer	Focus first	Right first focus	Tool-call field
Teacher	7	48%	1
SFT	1	32%	8
Ordinary RL, round 1	1	29%	9
Ordinary RL, round 2	2	37%	9
Ordinary RL, round 3	2	37%	10
LeAH, round 1	1	35%	5
LeAH, round 2	1	36%	6
LeAH, round 3	5	46%	0

Table 12: Final edits, of 9 per benchmark and proposer, that use a rule specific to the benchmark.

Benchmark	Rule specific to the benchmark	LeAH	Ordinary RL	SFT	Teacher
ScienceWorld	Focus on the task's object first, or in the order the task gives	5	2	1	7
BabyAI	A goal object that is not in view is in another room, so open the door and go through it	5	2	2	4
BIRD	Take each column from the table that holds it	3	2	1	3
TextWorld	When the game rejects a command or asks about it, change the form of the command	2	1	0	8
Gridworld	A direction in the feedback holds only for the room it names	2	0	0	3
Total, of 45		17	7	4	25

LeAH leads on ScienceWorld where its edits state the rule. In its five edits with the rule, the first focus is right in 55% of the episodes, against 36% in its four other edits and 37% over all edits of ordinary reinforcement learning. Once the first focus is right, the episodes of the three proposers reach about the same score (63.1, 62.6 and 64.9 for LeAH, ordinary reinforcement learning and the SFT proposer), so the 41 additional episodes of LeAH with a right first focus account for most of its 6.3-point lead on ScienceWorld.

Two more ScienceWorld episodes from the context of Figure 3a show the same difference on a different kind of task. The task reads "Your task is to find a(n) non-living thing. First, focus on the thing. Then, move it to the yellow box in the bathroom." Under LeAH's edit, the executor focuses on the red apple in its first turn, carries it to the bathroom and puts it in the yellow box, scoring 100 in six turns. Under ordinary reinforcement learning's edit, the executor moves the apple to the yellow box without focusing on it, then waits and looks around until the step limit, scoring 25.

BabyAI, BIRD, TextWorld and Gridworld also have rules of their own, and LeAH's edits use them more often than those of ordinary reinforcement learning on each of them (Table 12).

On BabyAI, when the goal object is not in view, LeAH's edits tell the executor that it is in another room and that it should open the door and go through it. On a task to go to a grey ball two rooms away, the executor under LeAH's edit goes through both doors and reaches the ball in three steps, while under the edit of ordinary reinforcement learning it goes through the first door, decides "Since the door is closed and I don't have a key . . . , I cannot open it yet" and turns in place until the episode ends.

On TextWorld, the game rejects a command it cannot parse and asks a question when a command is incomplete, such as "What do you want to cook the red apple with?" LeAH's edit tells the executor not to resend a rejected command in the same form, and the executor answers `cook apple on stove`, then prepares and eats the meal (score 100). Ordinary reinforcement learning's edit for the same context reads the question as a missing tool, "take the required tool or appliance and try again", and the executor repeats `get stove`, which the game answers with "That's fixed in place." (score 25).

H.3 Where the Hooks Read the Action

On the training benchmarks, the executor acts through native tool calls, so a hook can read the action either from the tool-call field of the executor's reply or from its own arguments. On ScienceWorld, BabyAI, TextWorld and Gridworld, the executor replies in plain text, the harness parses the action from the text and passes it to `before_tool` and `after_tool` as an argument and the tool-call field stays empty. A hook that reads only the tool-call field therefore records no action on these benchmarks, and a warning built on it never appears.

None of LeAH's 36 final edits for these four benchmarks reads the tool-call field, against 10 of ordinary reinforcement learning's, 8 of the SFT proposer's and 1 of the teacher's. For example, ordinary reinforcement learning's ScienceWorld edit for one context collects the executor's actions only from that field and warns when an action repeats, so the warning never appears, and two of its BabyAI edits build their warnings against loops the same way. On the TextWorld context of Figure 3b, the four edits whose hooks read the argument, LeAH's three and one of ordinary reinforcement learning's, score from 40.3 to 49.9, while the two edits of ordinary reinforcement learning that read the tool-call field score 33.7 and 37.8. In the episode of Figure 3b, LeAH's warning appears after turns 4, 6, 12 and 18, and the executor leaves the loop at turn 19, while under ordinary reinforcement learning's edit the executor alternates `go south` and `go north` 50 times each, until the step limit of 100 turns.

H.4 Over Meta-Training

Table 11 follows the two findings over the three rounds. LeAH's rule about the focus and its rise in right first focus both appear after the third round (in 1, 1 and 5 edits, and in 35%, 36% and 46% of the episodes), when its hooks also stop reading the tool-call field (5, 6 and 0 edits). Ordinary reinforcement learning reaches 37% after the second round and stays there, and keeps the SFT proposer's habit of reading the tool-call field (9, 9 and 10 edits, against 8 for the SFT proposer).

A transfer edit is an edit that the proposer writes for one training benchmark after reading an edit for another. Within the same pair of source and target benchmarks, a first-round transfer edit that copies the system prompt of its source solves 4.2 fewer of the 50 tasks on DBBench and 8.1 fewer on WebShop than one that does not. Over the three rounds, the share of transfer edits that carry the interface of their source, through its system prompt, tool names or argument names, falls from 20.8% to 9.4% and 7.3%, and the share that copies the source's system prompt falls from 8.3% to 2.4% and 1.1%. The transfer edits that succeed restate the fix in the terms of the target. One ALFWorld candidate carries its rule against repeating an action to WebShop with the exception that WebShop needs, "Clicking an option may leave the page text looking unchanged; that is normal, so do not go back to search because of it", and its transfer edit raises the WebShop context from 11 to 25 of 50 tasks. The transfer step thus gives the proposer practice in writing a fix for a benchmark other than the one it came from, and rewards the rewrites that fit the target.

I Additional Related Work

Reinforcement learning for self-improvement. Self-improvement methods use model-generated data or feedback to train task policies, and can also learn decisions about adaptation. ReST trains on generated translations selected by a learned reward model, and ReST^{EM} on generated solutions selected by their correctness (Gulcehre et al., 2023; Singh et al., 2024). Other methods use language model evaluations, model scores or agreement among sampled answers as feedback (Lee et al., 2024; Pang et al., 2024; Wang et al., 2025b; Prasad et al., 2025; Zuo et al., 2025). These approaches learn from feedback on task outputs. Transformer-Squared trains task-specific

weight adjustments with reinforcement learning (Sun et al., 2025), SEAL rewards self-edits by performance after fine-tuning (Zweiger et al., 2025) and SCoL learns which layers to update from incoming context (Wang et al., 2026c). In these methods, the learned action changes the model's own parameters, and in SEAL and SCoL each committed update changes the model that selects the next one. Our proposer instead edits the harness of a separate executor, and at test time it writes an edit while both models' weights remain fixed.

Meta-learning for optimization. Learning to optimize meta-trains an optimizer on a distribution of problems so that its update rule improves new ones (Andrychowicz et al., 2016; Li & Malik, 2017), and learned optimizers are judged by how far this rule generalizes beyond the problems it was trained on (Wichrowska et al., 2017; Metz et al., 2022). Language models also act as optimizers that revise prompts and solutions from feedback (Yang et al., 2024; Zhou et al., 2023). Our proposer is a learned harness optimizer: it maps the failures of a harness to an edit, and LeAH meta-trains this adaptation rule across benchmarks so that it generalizes to new ones.